

DATABRICKS MILESTONE 2

350 UNIQUE MCQs + SCENARIO-BASED PRACTICE

200 Direct Concept / Code-Interpretation Questions
150 Scenario-Based Questions

Rebuilt version

The previous PDF repeated concepts in lightly reworded pairs. This version removes that pattern and uses varied question structures: concepts, code interpretation, comparisons, API selection, debugging, performance diagnosis and realistic scenarios.

Quality controls

- 350 numbered questions total
- 200 direct + 150 scenario-based
- Duplicate question-text check passed
- Adjacent duplicate-pattern check passed
- Correct answers distributed across A/B/C/D
- Content grounded in the uploaded Milestone 2 preparation handbook

Answer distribution: A=88 | B=88 | C=87 | D=87

SECTION A — 200 DIRECT MCQs

Concept recall, code interpretation, API selection, comparisons, output prediction, debugging and performance choices.

1. A Java source file is passed to javac. What artifact is normally produced for execution by the JVM?

Code pipeline • Java/JVM

- A. .class bytecode
- B. .java source only
- C. JDBC ResultSet
- D. serialized object

Answer: A. .class bytecode

Why: javac compiles Java source into bytecode that the JVM can execute.

2. Which component actually executes Java bytecode?

Component identification • Java/JVM

- A. JDK
- B. JVM
- C. javac
- D. JDBC driver

Answer: B. JVM

Why: The JVM is the runtime that executes bytecode.

3. A developer needs compiler and development tools, not merely a runtime. Which package concept fits?

Requirement mapping • Java/JVM

- A. JVM
- B. JRE alone
- C. JDK
- D. HttpClient

Answer: C. JDK

Why: The JDK contains development tools; the JVM executes bytecode.

4. A method creates an object with new. Which memory distinction is the most accurate at this assessment level?

Concept distinction • Java/JVM

- A. all references are separate heap objects
- B. all local variables are heap objects
- C. objects are always stored on the stack
- D. the object is generally on the heap while the method frame is on the stack

Answer: D. the object is generally on the heap while the method frame is on the stack

Why: Method stack frames hold locals/references; created objects are generally on the heap.

5. Which conversion is autoboxing?

Direction test • Java wrappers

- A. int -> Integer
- B. Integer -> int
- C. String -> Integer
- D. double -> float

Answer: A. int -> Integer

Why: Autoboxing converts a primitive to its wrapper.

6. Which conversion is unboxing?

Direction test • Java wrappers

- A. int -> Integer
- B. Integer -> int
- C. String -> Integer
- D. Object -> String

Answer: B. Integer -> int

Why: Unboxing converts a wrapper to its primitive.

7. A method has a local variable named x and an object field also named x. Which keyword explicitly refers to the current object's field?

Code reading • Java OOP

- A. super.x
- B. class.x
- C. this.x
- D. current.x

Answer: C. this.x

Why: this refers to the current object and disambiguates its field.

8. A subclass must invoke a constructor in its immediate parent. Which constructor call is appropriate?

Constructor selection • Java OOP

- A. this(...)
- B. parent(...)
- C. base(...)
- D. super(...)

Answer: D. super(...)

Why: super(...) invokes a parent constructor; this(...) targets another constructor in the same class.

9. A class needs one constructor to delegate to another constructor in the same class. Which call is used?

Constructor selection • Java OOP

- A. this(...)
- B. super(...)
- C. delegate(...)
- D. self(...)

Answer: A. this(...)

Why: this(...) invokes another constructor in the same class.

10. Which property identifies a Java constructor?

Definition with traps • Java OOP

- A. same class name with void
- B. same class name and no return type
- C. any method name with no parameters
- D. static method with class name

Answer: B. same class name and no return type

Why: Constructors have the class name and no return type.

11. Two methods have the same name but different parameter lists. What mechanism is being used?

Compare concepts • Java OOP

- A. overriding
- B. field hiding
- C. overloading
- D. composition

Answer: C. overloading

Why: Overloading uses the same method name with different parameter lists and is resolved at compile time.

12. A parent reference points to a child object and an overridden instance method is called. Which implementation is selected?

Runtime reasoning • Java OOP

- A. the parent's implementation always
- B. the method chosen by return type
- C. the constructor implementation
- D. the child's overridden implementation

Answer: D. the child's overridden implementation

Why: Overridden instance behavior uses runtime dispatch.

13. A child overrides a public method. Can it make the method private?

Rule application • Java OOP

- A. No; overriding cannot narrow visibility
- B. Yes, visibility is unrelated
- C. Yes, if parameters are unchanged
- D. Only if the parent method is static

Answer: A. No; overriding cannot narrow visibility

Why: An overriding method cannot reduce the inherited method's visibility.

14. Which design best represents composition?

Design pattern • Java OOP

- A. Car extends Engine
- B. Car contains an Engine object
- C. Engine extends Car
- D. Car has only static fields

Answer: B. Car contains an Engine object

Why: Composition models a has-a relationship through contained/used objects.

15. Which statement best characterizes an interface at this level?

Definition • Java OOP

- A. a class that must be instantiated
- B. a database table
- C. a contract implemented by classes
- D. a constructor template

Answer: C. a contract implemented by classes

Why: An interface defines a contract implemented by classes.

16. Why can an abstract class be useful when several classes share state and some behavior?

Use-case • Java OOP

- A. it can only contain constants
- B. it must be directly instantiated
- C. it removes inheritance
- D. it can provide shared implementation/state plus abstract methods

Answer: D. it can provide shared implementation/state plus abstract methods

Why: Abstract classes can provide shared state/implementation and abstract methods.

17. A field is private and can only be changed through validated methods. Which OOP principle is most directly illustrated?

Principle identification • Java OOP

- A. encapsulation
- B. polymorphism
- C. overloading
- D. composition

Answer: A. encapsulation

Why: Encapsulation hides state and controls updates through operations.

18. What does final on an instance method prevent?

Keyword effect • Java OOP

- A. calling the method
- B. overriding the method
- C. inheriting the class
- D. passing parameters

Answer: B. overriding the method

Why: A final method cannot be overridden.

19. A class should have one primary reason to change. Which design principle is this?

Design principle • Java OOP

- A. Open/Closed Principle
- B. DRY only
- C. Single Responsibility Principle
- D. dependency inversion

Answer: C. Single Responsibility Principle

Why: SRP focuses on one primary responsibility/reason to change.

20. Which reference can legally point to several concrete implementations of the same abstraction?

Polymorphism • Java OOP

- A. only a primitive reference
- B. a constructor reference
- C. a private field reference
- D. a parent/interface reference

Answer: D. a parent/interface reference

Why: Polymorphism allows a parent/interface reference to refer to different implementations.

21. A collection must preserve insertion order and permit duplicate values. Which interface is the natural fit?

Requirement mapping • Java Collections

- A. List
- B. Set
- C. Map
- D. ConcurrentHashMap

Answer: A. List

Why: List models an ordered sequence and permits duplicates.

22. A collection must enforce uniqueness of elements. Which abstraction is the best fit?

Requirement mapping • Java Collections

- A. List
- B. Set
- C. Map
- D. String

Answer: B. Set

Why: Set models uniqueness.

23. A lookup table maps customerId to customerName. Which abstraction directly models it?

Requirement mapping • Java Collections

- A. List
- B. Set
- C. Map
- D. Iterator

Answer: C. Map

Why: Map stores key-value associations.

24. A HashMap receives a second put using an equal existing key. What happens to the mapping?

Behavior prediction • Java HashMap

- A. a second equal key is stored
- B. the whole map is cleared
- C. the new value is rejected
- D. the existing key's value is replaced

Answer: D. the existing key's value is replaced

Why: An equal existing key maps to the newly supplied value.

25. For logical uniqueness in a HashSet, which pair is important?

Mechanism identification • Java HashSet

- A. consistent equals() and hashCode()
- B. toString() and clone()
- C. insertion order and size()
- D. only object identity

Answer: A. consistent equals() and hashCode()

Why: Hash-based collections rely on hashCode and equals for logical uniqueness.

26. What is the primary benefit of Java generics such as List?

Purpose • Java Generics

- A. automatic thread synchronization
- B. compile-time type safety
- C. automatic serialization
- D. runtime-only typing

Answer: B. compile-time type safety

Why: Generics provide parameterized compile-time type safety and reduce unsafe casts.

27. For List, what can the unknown element type be?

Wildcard reasoning • Java Generics

- A. only Integer
- B. only Number and never subclasses
- C. Number or an unknown subtype of Number
- D. any unrelated type

Answer: C. Number or an unknown subtype of Number

Why: ? extends Number permits Number or an unknown subtype.

28. An iterator has just returned an element with next(). Which operation removes that returned element through the iterator?

API selection • Java Iterator

- A. iterator.delete()
- B. collection.clearOne()
- C. removeNext()
- D. iterator.remove()

Answer: D. iterator.remove()

Why: Iterator.remove removes the last element returned by next in a coordinated way.

29. What is the result type of byte + byte in Java arithmetic?

Type promotion • Java Numeric

- A. int
- B. byte
- C. short
- D. long

Answer: A. int

Why: byte and short operands are promoted to int in arithmetic.

30. Given false && expensiveCheck(), what happens to expensiveCheck()?

Evaluation tracing • Java Short-circuit

- A. it always runs
- B. it is skipped
- C. only its return value is skipped
- D. the expression becomes true before evaluation

Answer: B. it is skipped

Why: && skips the RHS when the LHS is false.

31. Given true || expensiveCheck(), what happens to expensiveCheck()?

Evaluation tracing • Java Short-circuit

- A. it always runs
- B. only its arguments run
- C. it is skipped
- D. the expression becomes false

Answer: C. it is skipped

Why: || skips the RHS when the LHS is true.

32. What is the value of 2 + 3 * 4 in Java?

Code output • Java Precedence

- A. 20
- B. 24
- C. 9
- D. 14

Answer: D. 14

Why: Multiplication has higher precedence than addition.

33. A class is Serializable and declares transient String token. What happens to token under default serialization?

Serialization behavior • Java Serialization

- A. it is excluded from the default serialized state
- B. it is always encrypted
- C. it becomes static
- D. it is serialized as a database key

Answer: A. it is excluded from the default serialized state

Why: Transient fields are skipped by default Java serialization.

34. What is the purpose of serialVersionUID?

Purpose • Java Serialization

- A. database primary key
- B. serialization compatibility/version identifier
- C. memory address
- D. security token

Answer: B. serialization compatibility/version identifier

Why: It identifies the serialization version for compatibility checks.

35. A program must write raw image bytes. Which Java I/O family is appropriate?

API selection • Java IO

- A. PrintWriter only
- B. Pattern/Matcher
- C. byte streams such as FileInputStream/FileOutputStream
- D. java.time

Answer: C. byte streams such as FileInputStream/FileOutputStream

Why: Byte streams handle binary/raw bytes.

36. Why is try-with-resources preferred for a JDBC Connection or file stream?

Reasoning • Java IO

- A. it automatically commits a transaction
- B. it creates a new thread
- C. it infers a Spark schema
- D. it automatically closes the managed resource

Answer: D. it automatically closes the managed resource

Why: try-with-resources closes AutoCloseable resources when the block exits.

37. Which Java API is intended for declarative processing of collection elements with operations such as filter/map?

API identification • Java APIs

- A. Streams API
- B. JVM bytecode compiler
- C. FileInputStream
- D. JDBC ResultSet

Answer: A. Streams API

Why: Java Streams provide declarative collection processing.

38. Which regex object represents the compiled pattern?

API roles • Java Regex

- A. Matcher
- B. Pattern
- C. ResultSet
- D. Scanner

Answer: B. Pattern

Why: Pattern compiles a regular expression; Matcher applies it to input.

39. Which Matcher operation is suited to validating that the entire input matches a regex?

Method choice • Java Regex

- A. find()
- B. group()
- C. matches()
- D. appendReplacement()

Answer: C. matches()

Why: matches() attempts whole-input matching; find() searches for a subsequence.

40. Which java.time type includes date, time, and time-zone context?

Type selection • Java Time

- A. LocalDate
- B. LocalTime
- C. LocalDateTime
- D. ZonedDateTime

Answer: D. ZonedDateTime

Why: ZonedDateTime carries date/time plus zone context.

41. What is the normal flow for Java HttpClient?

API flow • Java HTTP

- A. client.send(request) -> response
- B. response.send(client)
- C. request executes itself
- D. URI becomes a response

Answer: A. client.send(request) -> response

Why: HttpClient sends an HttpRequest and receives an HttpResponse.

42. What does Files.walk(path) provide conceptually?

API behavior • Java Files

- A. a JDBC transaction
- B. a stream for directory-tree traversal
- C. a regex compiler
- D. an HTTP response

Answer: B. a stream for directory-tree traversal

Why: Files.walk traverses a directory tree and returns a stream of paths.

43. Which declaration can be rebound after initialization?

Syntax semantics • Scala Basics

- A. val
- B. tuple
- C. var
- D. List

Answer: C. var

Why: val is a fixed binding; var can be reassigned.

44. What does Scala commonly infer when a variable is initialized without an explicit type?

Type inference • Scala Basics

- A. the JVM ignores the type
- B. the type is inferred only at runtime
- C. every type must be written explicitly
- D. the compiler infers the type from the expression

Answer: D. the compiler infers the type from the expression

Why: Scala's compiler can infer many types from assigned expressions.

45. Which value best represents a fixed heterogeneous group such as (42, "IN", true)?

Data structure • Scala Basics

- A. a tuple
- B. a homogeneous Array only
- C. a Map with one key
- D. a Set only

Answer: A. a tuple

Why: Tuples group a fixed number of values that may have different types.

46. Which statement best contrasts Scala Array and immutable List?

Comparison • Scala Basics

- A. both are immutable and unindexed
- B. Array is indexed/mutable; immutable List is not updated in place
- C. List is always mutable while Array is not indexed
- D. Array is only for key-value data

Answer: B. Array is indexed/mutable; immutable List is not updated in place

Why: Scala Array is indexed and mutable; immutable List derives new values rather than mutating in place.

47. What does `x :: xs` do for an immutable Scala List?

Operator behavior • Scala Basics

- A. mutates xs in place
- B. removes the head
- C. creates a new List with x at the front
- D. appends x at the end

Answer: C. creates a new List with x at the front

Why: `::` prepends and returns a new immutable List.

48. In Scala pattern matching, which case is considered first when multiple cases could match?

Control flow • Scala Basics

- A. a random matching case
- B. the last case only
- C. all cases simultaneously
- D. the first matching case in order

Answer: D. the first matching case in order

Why: match evaluates cases in order and selects the first applicable case.

49. For case x if $x > 10 \Rightarrow \dots$, what must be true before the case is selected?

Guard reasoning • Scala Basics

- A.** the pattern matches and the guard is true
- B.** only the guard matters
- C.** the pattern must fail
- D.** the guard is evaluated after all cases

Answer: A. the pattern matches and the guard is true

Why: A guard adds an if-condition to a matching case.

50. What is $x \Rightarrow x * 2$?

Syntax identification • Scala Basics

- A.** a Spark action
- B.** an anonymous function value
- C.** a class declaration
- D.** a JDBC query

Answer: B. an anonymous function value

Why: It is a lambda/function value.

51. If `xs = List(2,4,6)`, what does `xs.map(_ / 2)` conceptually produce?

Code output • Scala Functional

- A. `List(2,4,6)`
- B. `List(1,4,9)`
- C. `List(1,2,3)`
- D. `List(3,5,7)`

Answer: C. `List(1,2,3)`

Why: `map` applies one transformation to each input element.

52. Which operation keeps only elements satisfying a predicate?

API selection • Scala Functional

- A. `map`
- B. `flatMap`
- C. `reduceByKey`
- D. `filter`

Answer: D. `filter`

Why: `filter` retains elements satisfying a predicate.

53. Which operation is appropriate when each input can produce zero or more outputs and nested results should be flattened?

Requirement mapping • Scala Functional

- A. `flatMap`
- B. `map`
- C. `filter`
- D. `sorted`

Answer: A. `flatMap`

Why: `flatMap` maps and flattens nested results.

54. A function that accepts A and returns Boolean has which common predicate type?

Type signature • Scala Functional

- A. `Boolean => A`
- B. `A => Boolean`
- C. `A => A`
- D. `() => A`

Answer: B. `A => Boolean`

Why: Predicates commonly have type `A => Boolean`.

55. Why are pure functions easier to reason about in distributed processing?

Reasoning • Scala Functional

- A. they must mutate global state
- B. they depend on the clock
- C. their result depends on explicit inputs without hidden side effects
- D. they require a database

Answer: C. their result depends on explicit inputs without hidden side effects

Why: Purity improves determinism and reduces hidden effects.

56. What is a higher-order function?

Definition • Scala Functional

- A. a function that only returns `String`
- B. a function that only accepts `Int`
- C. a mutable variable
- D. a function that accepts or returns a function

Answer: D. a function that accepts or returns a function

Why: Higher-order functions accept a function, return one, or both.

57. A pipeline should first select valid rows and then transform them. Which order is usually most direct?

Pipeline design • Scala Functional

- A. filter then map
- B. map then filter always
- C. flatMap then constructor only
- D. sort then filter only

Answer: A. filter then map

Why: Filtering first can reduce the number of elements passed to later work when logically valid.

58. What does immutability favor?

Design reasoning • Scala Functional

- A. global mutation
- B. deriving new values rather than changing shared state in place
- C. reassigning val
- D. shared mutable state

Answer: B. deriving new values rather than changing shared state in place

Why: Immutable style creates new values instead of mutating shared state.

59. Which pipeline most directly expands each word into its characters and flattens the result?

Code selection • Scala Functional

- A. words.filter(_.nonEmpty)
- B. words.map(_.length)
- C. words.flatMap(w => w.toList)
- D. words.sorted

Answer: C. words.flatMap(w => w.toList)

Why: flatMap can create nested collections and flatten them.

60. If a List contains three elements, how many outputs does map normally produce?

Cardinality • Scala Functional

- A. zero always
- B. one total
- C. unbounded without relation to inputs
- D. three

Answer: D. three

Why: map produces one output per input element.

61. Which operation is least appropriate if the requirement is only to remove values below a threshold?

Trap recognition • Scala Functional

- A. flatMap
- B. filter
- C. map
- D. takeWhile

Answer: A. flatMap

Why: filter is the direct operation for retaining elements that satisfy a predicate.

62. A val points to an immutable List. What does immutability mean for an operation that 'adds' an element?

Semantics • Scala Functional

- A. the original List is changed in place
- B. a new List is derived rather than the original List being mutated
- C. val becomes reassignable automatically
- D. the JVM edits the List

Answer: B. a new List is derived rather than the original List being mutated

Why: Immutable collections are transformed into new values.

63. A local Scala Seq is already in driver memory and needs to become an RDD for a small test. Which API fits?

Creation path • Spark RDDs

- A. `textFile`
- B. `read.jdbc`
- C. `parallelize`
- D. `createDataFrame`

Answer: C. `parallelize`

Why: `parallelize` converts a driver-resident local collection into an RDD.

64. A large text file is an external source. Which basic SparkContext operation is appropriate for an RDD?

Creation path • Spark RDDs

- A. `parallelize`
- B. `collect`
- C. `cache`
- D. `textFile`

Answer: D. `textFile`

Why: `textFile` creates an RDD from an external text source.

65. What is the defining execution property of an RDD transformation such as `filter`?

Execution model • Spark RDDs

- A. it is lazy
- B. it immediately returns all records to the driver
- C. it always writes a file
- D. it commits a JDBC transaction

Answer: A. it is lazy

Why: Transformations build lineage and wait for an action to execute.

66. Which operation triggers Spark execution?

Classification • Spark RDDs

- A. `filter(...)`
- B. `count()`
- C. `map(...)`
- D. `flatMap(...)`

Answer: B. `count()`

Why: `count` is an action; `map/filter/flatMap` are transformations.

67. Why is `collect()` risky on a very large RDD?

Failure reasoning • Spark RDDs

- A. it deletes the RDD
- B. it always shuffles every partition twice
- C. it brings the entire RDD to the driver
- D. it writes only five records

Answer: C. it brings the entire RDD to the driver

Why: `collect` returns all elements to driver memory.

68. If only a small sample is needed from a large RDD, which action is more appropriate than `collect()`?

API selection • Spark RDDs

- A. `collect()`
- B. `reduceByKey()`
- C. `cache()`
- D. `take(5)`

Answer: D. `take(5)`

Why: `take` can retrieve a small number of elements without materializing the entire RDD.

69. Which statement best describes lazy evaluation?

Concept • Spark RDDs

- A. Spark delays transformations until an action requires execution
- B. every transformation immediately runs
- C. actions are never executed
- D. only JDBC reads are lazy

Answer: A. Spark delays transformations until an action requires execution

Why: Spark delays transformation execution until an action.

70. What does RDD lineage represent?

Fault tolerance • Spark RDDs

- A. a copy of every row
- B. the dependency history of transformations used to derive the RDD
- C. only cached bytes
- D. a JDBC connection list

Answer: B. the dependency history of transformations used to derive the RDD

Why: Lineage records transformation dependencies and supports recomputation.

71. If a lost partition can be recomputed from earlier transformations, which RDD property enables that idea?

Recovery • Spark RDDs

- A. Map keys
- B. driver heap only
- C. lineage
- D. regex Pattern

Answer: C. lineage

Why: Lineage allows Spark to reconstruct lost partitions from dependencies.

72. Which is typically a narrow transformation?

Classification • Spark RDDs

- A. groupByKey
- B. repartition
- C. a broad join
- D. map

Answer: D. map

Why: map/filter typically have limited partition dependencies and avoid broad redistribution.

73. Which operation commonly requires a shuffle because values must be grouped by key?

Shuffle recognition • Spark Pair RDDs

- A. groupByKey
- B. mapValues
- C. map
- D. filter

Answer: A. groupByKey

Why: Grouping by key generally redistributes data across partitions.

74. A pair RDD contains (region, amount). Which operation directly computes a total per region?

Aggregation selection • Spark Pair RDDs

- A. collect()
- B. reduceByKey(_ + _)
- C. filter(_._2 > 0)
- D. take(10)

Answer: B. reduceByKey(_ + _)

Why: reduceByKey combines values for each key.

75. What must exist before reduceByKey can be applied conceptually?

Precondition • Spark Pair RDDs

- A. only Strings
- B. a DataFrame schema
- C. key-value pairs
- D. a JDBC Connection

Answer: C. key-value pairs

Why: Pair RDD aggregations operate on (key,value) records.

76. Which pair RDD operation combines records sharing a key across two pair RDDs?

API selection • Spark Pair RDDs

- A. collect
- B. map
- C. take
- D. join

Answer: D. join

Why: join combines matching keys from two pair RDDs.

77. What does keys() conceptually return from a pair RDD?

API behavior • Spark Pair RDDs

- A. the key component of each pair
- B. only unique keys regardless of data
- C. the values only
- D. the full tuple plus schema

Answer: A. the key component of each pair

Why: keys extracts the key component.

78. What does values() conceptually return from a pair RDD?

API behavior • Spark Pair RDDs

- A. only keys
- B. the value component of each pair
- C. partition IDs
- D. the entire RDD as a driver array

Answer: B. the value component of each pair

Why: values extracts the value component.

79. Why can reduceByKey be preferable to groupByKey for a simple sum?

Comparison • Spark Pair RDDs

- A. it never partitions data
- B. it always runs on the driver
- C. it combines values per key using the reduction function instead of materializing all values for the group
- D. it converts the RDD to JSON

Answer: C. it combines values per key using the reduction function instead of materializing all values for the group

Why: The handbook emphasizes reduceByKey for associative/commutative per-key aggregation.

80. What does cache() primarily aim to improve?

Purpose • Spark RDDs

- A. schema inference
- B. JDBC commit durability
- C. regex matching
- D. reuse of a computed RDD across later actions

Answer: D. reuse of a computed RDD across later actions

Why: Caching/persistence can avoid recomputing reused data.

81. When is persisting an RDD especially useful?

Scenario selection • Spark RDDs

- A. when the same computed RDD is reused by multiple downstream actions
- B. when the RDD is never reused
- C. when only a single literal is needed
- D. when no action will ever run

Answer: A. when the same computed RDD is reused by multiple downstream actions

Why: Persistence helps repeated reuse of computed data.

82. What is the conceptual distinction between persistence and checkpointing?

Comparison • Spark RDDs

- A. both mean exactly the same thing
- B. persistence speeds reuse; checkpointing creates durable recovery-oriented state/data
- C. checkpointing is only a filter
- D. persistence commits JDBC transactions

Answer: B. persistence speeds reuse; checkpointing creates durable recovery-oriented state/data

Why: The handbook distinguishes reuse optimization from recovery/progress/state durability.

83. If an RDD pipeline contains only map and filter and no action, what has happened?

Execution tracing • Spark RDDs

- A. all data has already been collected
- B. the job has committed
- C. the lineage has been defined but full execution has not been triggered
- D. the driver has written a file

Answer: C. the lineage has been defined but full execution has not been triggered

Why: Transformations are lazy until an action triggers execution.

84. Which operation is an action rather than a transformation?

Classification • Spark RDDs

- A. map()
- B. filter()
- C. flatMap()
- D. reduce()

Answer: D. reduce()

Why: reduce triggers execution and returns an aggregated result.

85. A developer wants distributed output rather than all data on the driver. Which category should they consider?

Design choice • Spark RDDs

- A. a save/write action
- B. collect() only
- C. toString()
- D. hashCode()

Answer: A. a save/write action

Why: Save/write actions produce distributed output instead of returning all records to the driver.

86. What is the main caution when using parallelize?

Common trap • Spark RDDs

- A. it only accepts JDBC tables
- B. the source collection is driver-resident, so it is mainly suited to small examples/testing
- C. it always creates one partition
- D. it requires a regex

Answer: B. the source collection is driver-resident, so it is mainly suited to small examples/testing

Why: The handbook flags parallelize as driver-resident input mainly for small examples/testing.

87. Which operation changes each element independently without requiring key redistribution in the typical case?

Partition reasoning • Spark RDDs

- A. groupByKey
- B. reduceByKey
- C. map
- D. repartition

Answer: C. map

Why: map is a typical narrow transformation.

88. Which operation is designed to keep elements satisfying a predicate?

API selection • Spark RDDs

- A. reduceByKey
- B. join
- C. collect
- D. filter

Answer: D. filter

Why: filter retains records that satisfy the predicate.

89. If a line contains several words and each word should become a separate RDD element, which transformation is natural?

Transformation choice • Spark RDDs

- A. flatMap
- B. map only
- C. reduceByKey
- D. cache

Answer: A. flatMap

Why: flatMap can emit multiple outputs per input and flatten them.

90. A pair RDD has duplicate keys and values 4 and 6 for key A. What does reduceByKey(_ + _) produce for A?

Numeric code reasoning • Spark Pair RDDs

- A. A -> 24
- B. A -> 10
- C. A -> 2
- D. A -> 46

Answer: B. A -> 10

Why: The reduction combines the values for A: $4 + 6 = 10$.

91. If `rdd = sc.parallelize(Seq(1,2,3,4))`, what does `rdd.filter(_ % 2 == 0)` retain?

Code output • Spark RDDs

- A. 1 and 3
- B. all four
- C. 2 and 4
- D. only 4

Answer: C. 2 and 4

Why: filter retains values satisfying the even predicate.

92. If `rdd = sc.parallelize(Seq(1,2,3))`; `rdd.map(_ * 10)`, how many output elements exist?

Cardinality • Spark RDDs

- A. one
- B. two
- C. six
- D. three

Answer: D. three

Why: map produces one output per input.

93. What is a practical reason to avoid collect() for hundreds of GB?

Operational reasoning • Spark RDDs

- A. driver memory can be overwhelmed
- B. the cluster cannot read files
- C. collect disables lineage permanently
- D. collect changes keys

Answer: A. driver memory can be overwhelmed

Why: All elements are brought to the driver, making large collects unsafe.

94. A join of two pair RDDs suddenly shows heavy data movement. Which concept best explains it?

Performance diagnosis • Spark Pair RDDs

- A. autoboxing
- B. shuffle/data redistribution
- C. constructor chaining
- D. regex matching

Answer: B. shuffle/data redistribution

Why: Key-based joins commonly require redistribution of records.

95. Which statement about transformations is most accurate?

Concept • Spark RDDs

- A. they always return a Java ArrayList
- B. they immediately execute every partition
- C. they define new RDDs lazily and contribute to lineage
- D. they automatically persist to disk

Answer: C. they define new RDDs lazily and contribute to lineage

Why: Transformations are lazy and build lineage.

96. Which action returns a bounded number of elements instead of all elements?

API selection • Spark RDDs

- A. collect()
- B. groupByKey()
- C. map()
- D. take(n)

Answer: D. take(n)

Why: take retrieves a limited number of elements.

97. For structured records where SQL-style operations and optimizer-friendly processing are desired, which Spark abstraction is generally preferred?

Abstraction choice • Spark DataFrames

- A. DataFrame
- B. raw RDD only
- C. Iterator only
- D. Java Thread

Answer: A. DataFrame

Why: The handbook describes DataFrames as schema-aware and optimizer-friendly for structured data.

98. What does an explicit schema provide compared with relying entirely on inference?

Schema reasoning • Spark DataFrames

- A. automatic JDBC commits
- B. declared field names and data types
- C. RDD lineage deletion
- D. regex compilation

Answer: B. declared field names and data types

Why: An explicit schema defines the intended structure and types.

99. Why might a team choose an RDD instead of a DataFrame?

Trade-off • Spark DataFrames

- A. they want automatic SQL optimization only
- B. they need regex Pattern compilation
- C. they need lower-level control over distributed collection transformations
- D. they need JDBC rollback

Answer: C. they need lower-level control over distributed collection transformations

Why: RDDs are lower-level distributed collections; DataFrames are structured and optimizer-friendly.

100. A DataFrame column contains null ratings. Which predicate is designed to test null explicitly?

API correctness • Spark DataFrames

- A. `col("rating") == null`
- B. `col("rating") != None`
- C. `col("rating") > null`
- D. `col("rating").isNull()`

Answer: D. `col("rating").isNull()`

Why: Spark SQL null semantics are handled with `isNull()/isNotNull()`.

101. Which predicate selects only rows where rating exists?

API correctness • Spark DataFrames

- A. `col("rating").isNotNull()`
- B. `col("rating") != null`
- C. `col("rating") == None`
- D. `col("rating") > 0`

Answer: A. `col("rating").isNotNull()`

Why: `isNotNull` explicitly checks for a non-null value.

102. A DataFrame needs only columns `customer_id` and `amount`. Which operation expresses that projection?

API selection • Spark DataFrames

- A. `collect()`
- B. `select("customer_id", "amount")`
- C. `cache()`
- D. `reduceByKey()`

Answer: B. `select("customer_id", "amount")`

Why: `select` projects the requested columns.

103. A DataFrame must keep rows where `amount > 100`. Which operation directly expresses the requirement?

API selection • Spark DataFrames

- A. `collect()`
- B. `selectExpr("amount")`
- C. `filter(col("amount") > 100)`
- D. `cache()`

Answer: C. `filter(col("amount") > 100)`

Why: `filter` selects rows satisfying a predicate.

104. Which operation adds or replaces a DataFrame column using an expression?

API selection • Spark DataFrames

- A. `collect`
- B. `take`
- C. `reduceByKey`
- D. `withColumn`

Answer: D. `withColumn`

Why: `withColumn` creates or replaces a named column.

105. A dataset must be summarized by `crop_type` using average `moisture_level`. Which pattern fits?

Aggregation • Spark DataFrames

- A. `groupBy("crop_type").avg("moisture_level")`
- B. `filter("crop_type")`
- C. `collect("moisture_level")`
- D. `withColumn("crop_type")`

Answer: A. `groupBy("crop_type").avg("moisture_level")`

Why: Grouping followed by `avg` computes an average per crop type.

106. Why is `dropna(subset=["rating"])` useful when only rating validity matters?

Parameter reasoning • Spark DataFrames

- A. it converts all nulls to zero
- B. it drops rows with nulls in the specified subset rather than using every column
- C. it removes the rating column
- D. it creates a new partition column

Answer: B. it drops rows with nulls in the specified subset rather than using every column

Why: The subset limits the columns considered for null removal.

107. A DataFrame has a string timestamp column and the assessment requires TimestampType. What is the conceptual operation?

Type conversion • Spark DataFrames

- A. collect the column to the driver
- B. cache it only
- C. cast/convert the column to TimestampType
- D. use reduceByKey

Answer: C. cast/convert the column to TimestampType

Why: Casting converts the column to the requested Spark SQL data type.

108. A developer writes df.withColumn("x", expr) but forgets to assign or otherwise use the returned DataFrame. What is the key issue?

Bug diagnosis • Spark DataFrames

- A. Spark always mutates in place
 - B. withColumn is an action
 - C. expr automatically edits every DataFrame
 - D. DataFrame transformations return a new DataFrame rather than mutating the original in place
- Answer: D. DataFrame transformations return a new DataFrame rather than mutating the original in place**

Why: Spark DataFrame transformations are immutable-style and return a new DataFrame.

109. Which operation is typically used to combine rows from two DataFrames based on matching keys?

API selection • Spark DataFrames

- A. join
- B. cache
- C. withColumn only
- D. take

Answer: A. join

Why: join combines DataFrames using a join condition/keys.

110. A DataFrame join and aggregation are unexpectedly slow. Which diagnostic is most relevant?

Performance diagnosis • Advanced Spark

- A. toString() only
- B. df.explain()
- C. hashCode()
- D. Pattern.compile()

Answer: B. df.explain()

Why: explain() helps inspect logical/physical plans and identify redistribution/shuffle.

111. In a Spark physical plan, what does Exchange commonly suggest?

Plan interpretation • Advanced Spark

- A. JVM autoboxing
- B. file closure
- C. data redistribution/shuffle
- D. regex matching

Answer: C. data redistribution/shuffle

Why: Exchange commonly represents repartitioning/data movement.

112. Why can shuffle be a performance concern?

Performance reasoning • Advanced Spark

- A. it disables all schemas
- B. it only affects Java constructors
- C. it converts every row to JSON
- D. it involves data movement/redistribution rather than only local partition processing

Answer: D. it involves data movement/redistribution rather than only local partition processing

Why: The handbook notes shuffle is more expensive than local partition processing.

113. A built-in Spark SQL function can perform a string transformation. Why is it generally preferred over a UDF?

Optimization choice • Advanced Spark

- A. Spark can understand and optimize the built-in expression more effectively
- B. UDFs are always faster
- C. built-ins require a JDBC connection
- D. UDFs are mandatory for all columns

Answer: A. Spark can understand and optimize the built-in expression more effectively

Why: Built-ins expose semantics to Spark and are generally easier to optimize.

114. Only 10% of rows need an expensive custom UDF. What is a sensible ordering when logically valid?

Performance pattern • Advanced Spark

- A. run the UDF on every row first
- B. apply a selective built-in filter before the UDF
- C. collect all rows then UDF
- D. disable filtering

Answer: B. apply a selective built-in filter before the UDF

Why: Filtering first reduces the number of rows processed by the expensive UDF.

115. A JDBC table has a suitable numeric partition column and sensible bounds. Which set of options enables parallel reads conceptually?

Configuration selection • Advanced Spark/JDBC

- A. only fetchSize
- B. only queryTimeout
- C. partitionColumn, lowerBound, upperBound, and numPartitions
- D. only user/password

Answer: C. partitionColumn, lowerBound, upperBound, and numPartitions

Why: These options define conceptual partitioned JDBC reads.

116. Why should numPartitions not be chosen arbitrarily high for a database source?

Operational reasoning • Advanced Spark/JDBC

- A. it disables schema inference
- B. it forces one partition
- C. it turns JDBC into a regex engine
- D. it also limits concurrent JDBC partitions/connections and the database may not support excessive concurrency

Answer: D. it also limits concurrent JDBC partitions/connections and the database may not support excessive concurrency

Why: The handbook cautions against excessive partitions beyond database capacity.

117. Selecting only three needed columns from a Parquet dataset can trigger what optimization?

Optimization recognition • Advanced Spark

- A. column pruning
- B. constructor chaining
- C. JDBC rollback
- D. autoboxing

Answer: A. column pruning

Why: Parquet is columnar and selecting only needed columns reduces I/O.

118. Filtering on a Parquet partition column can enable what optimization?

Optimization recognition • Advanced Spark

- A. bytecode compilation
- B. partition pruning
- C. thread locking
- D. Map replacement

Answer: B. partition pruning

Why: Filtering partition columns can avoid reading irrelevant partitions.

119. Which raw-to-curated flow best matches the handbook's format guidance?

Architecture choice • Advanced Spark

- A. write every stage as raw binary
- B. use only JDBC for all stages
- C. ingest flexible JSON, validate/transform, then write columnar structured output such as Parquet
- D. convert everything directly to a Java object

Answer: C. ingest flexible JSON, validate/transform, then write columnar structured output such as Parquet

Why: JSON is convenient for raw/semi-structured ingestion; Parquet is efficient for downstream analytics.

120. For new DataFrame-style Spark streaming applications, which API is generally preferred?

API choice • Streaming

- A. DStreams
- B. plain Java threads
- C. JDBC ResultSet
- D. Structured Streaming

Answer: D. Structured Streaming

Why: The handbook prefers Structured Streaming for new DataFrame-style streaming applications.

121. Why should streaming checkpoint data be stored durably?

Failure recovery • Streaming

- A. so progress/state metadata can survive failures and the query can recover
- B. so every row becomes a JDBC key
- C. so regex patterns persist
- D. so windows stop advancing

Answer: A. so progress/state metadata can survive failures and the query can recover

Why: Durable checkpoints support recovery after process/driver failure.

122. A 10-minute window updates every 2 minutes. Which statement is correct?

Parameter distinction • Streaming

- A. 2 minutes is the window and 10 is the slide
- B. 10 minutes is the window length; 2 minutes is the slide interval
- C. both are slide intervals
- D. both define watermarks

Answer: B. 10 minutes is the window length; 2 minutes is the slide interval

Why: Window length is the event-history span; slide controls how often results advance.

123. What problem do watermarks help manage?

Concept • Streaming

- A. Java autoboxing
- B. JDBC connection reuse
- C. late events and bounded retention of event-time state
- D. RDD object identity

Answer: C. late events and bounded retention of event-time state

Why: Watermarks provide a lateness threshold and help bound state for event-time operations.

124. Why is a socket source usually treated as a demo source rather than a durable production source?

Source evaluation • Streaming

- A. it always writes Parquet
- B. it cannot produce strings
- C. it automatically checkpoints to JDBC
- D. it is ephemeral and not designed for durable replay/recovery like production sources

Answer: D. it is ephemeral and not designed for durable replay/recovery like production sources

Why: The handbook describes socket input as ephemeral/demo-oriented.

125. Which older streaming abstraction is RDD-based?

Comparison • Streaming

- A. DStream
- B. Structured Streaming
- C. DataFrame schema
- D. JDBC partition

Answer: A. DStream

Why: DStreams are the legacy RDD-based streaming abstraction.

126. Which streaming abstraction uses DataFrame/Dataset-style structured APIs?

Comparison • Streaming

- A. DStream
- B. Structured Streaming
- C. HashSet
- D. PrintWriter

Answer: B. Structured Streaming

Why: Structured Streaming uses structured APIs.

127. A stream query must retain state for event-time windows but avoid unbounded growth from very late records. Which mechanism is most directly relevant?

Scenario-to-concept • Streaming

- A. autoboxing
- B. serialVersionUID
- C. watermark
- D. HashMap

Answer: C. watermark

Why: Watermarks help bound event-time state and handle lateness.

128. A transaction has two updates and the second fails before commit. Which operation undoes the uncommitted work?

Transaction reasoning • JDBC

- A. commit()
- B. close() only
- C. pool()
- D. rollback()

Answer: D. rollback()

Why: rollback undoes uncommitted transaction changes.

129. What does commit() do conceptually?

Transaction semantics • JDBC

- A. makes the transaction changes permanent
- B. undoes all work
- C. closes the connection only
- D. creates a new partition

Answer: A. makes the transaction changes permanent

Why: commit makes transaction changes permanent.

130. A service opens many database connections repeatedly. What mechanism can reduce physical connection creation overhead?

Performance mechanism • JDBC

- A. regex Pattern
- B. connection pooling
- C. RDD lineage
- D. watermark

Answer: B. connection pooling

Why: Connection pooling reuses physical database connections.

131. Two threads both read counter=10, both compute 11, and both write 11. What is the core problem?

Failure diagnosis • Concurrency

- A. deadlock only
- B. successful transaction
- C. race condition/lost update
- D. regex mismatch

Answer: C. race condition/lost update

Why: The result depends on timing of shared mutable state updates.

132. Thread A locks X then waits for Y; Thread B locks Y then waits for X. What risk is present?

Failure diagnosis • Concurrency

- A. autoboxing
- B. safe iteration
- C. column pruning
- D. deadlock

Answer: D. deadlock

Why: Circular waiting for locks can cause threads to wait indefinitely.

133. Which simple design idea reduces the deadlock risk when multiple locks are required?

Prevention • Concurrency

- A. acquire multiple locks in a consistent global order
- B. acquire them randomly
- C. never release them
- D. collect all threads to the driver

Answer: A. acquire multiple locks in a consistent global order

Why: Consistent lock ordering avoids circular wait patterns.

134. Several threads need concurrent map access and updates. Which Java collection is designed for this use case?

API selection • Concurrency

- A. HashMap without coordination
- B. ConcurrentHashMap
- C. CopyOnWriteArrayList
- D. Pattern

Answer: B. ConcurrentHashMap

Why: ConcurrentHashMap is designed for safe concurrent map access/updates.

135. A listener list is read thousands of times but modified rarely. Which collection fits conceptually?

Workload matching • Concurrency

- A. ArrayList with constant writes
- B. HashMap
- C. CopyOnWriteArrayList
- D. PrintWriter

Answer: C. CopyOnWriteArrayList

Why: CopyOnWriteArrayList suits read-heavy, write-rarely workloads.

136. Why is CopyOnWriteArrayList less attractive for frequent writes?

Trade-off • Concurrency

- A. it cannot be read
- B. it always deadlocks
- C. it requires JDBC
- D. modification creates a fresh internal copy, making writes expensive

Answer: D. modification creates a fresh internal copy, making writes expensive

Why: The handbook notes copy-on-write makes frequent writes expensive.

137. What does CompletableFuture represent conceptually?

API purpose • Concurrency

- A. an asynchronous computation that can be chained or combined
- B. an RDD partition
- C. a file stream only
- D. a database schema

Answer: A. an asynchronous computation that can be chained or combined

Why: CompletableFuture represents async computation and supports composition.

138. Which pattern best describes CompletableFuture.supplyAsync(...).thenApply(...)?

Code interpretation • Concurrency

- A. start a JDBC transaction and roll it back
- B. perform asynchronous work and transform its result
- C. compile a regex
- D. persist an RDD

Answer: B. perform asynchronous work and transform its result

Why: supplyAsync performs async work and thenApply transforms the resulting value.

139. A team uses a normal HashMap from multiple threads with unsynchronized compound updates. Which concern should be raised first?

Risk identification • Concurrency

- A. column pruning
- B. Parquet partition pruning
- C. unsafe concurrent access/shared-state race
- D. watermark delay

Answer: C. unsafe concurrent access/shared-state race

Why: The concurrency chapter emphasizes thread-safe structures for concurrent map access.

140. Given List xs = List.of(1,2,3); which Stream operation keeps only 2 and 3?

Code selection • Java APIs

- A. map
- B. sorted
- C. collect
- D. filter

Answer: D. filter

Why: filter selects elements according to a predicate.

141. Given Pattern p = Pattern.compile("A\\d+"); which object applies p to input text?

API role • Java Regex

- A. Matcher
- B. PatternBuilder
- C. ResultSet
- D. HttpResponse

Answer: A. Matcher

Why: Matcher applies a compiled Pattern to input.

142. For input "abc123", regex "\\d+", which Matcher method can locate 123 without requiring the whole input to match?

Method behavior • Java Regex

- A. matches()
- B. find()
- C. compile()
- D. groupCountOnly()

Answer: B. find()

Why: find searches for a matching subsequence.

143. A ZonedDateTime is needed because the business event must retain its time-zone region. Which type should be chosen?

Requirement mapping • Java Time

- A. LocalDate
- B. LocalTime
- C. ZonedDateTime
- D. LocalDateTime

Answer: C. ZonedDateTime

Why: ZonedDateTime includes date/time and zone context.

144. A file-reading block should close its FileInputStream even when an exception occurs. Which construct is designed for this?

Resource safety • Java IO

- A. switch
- B. synchronized only
- C. pattern matching
- D. try-with-resources

Answer: D. try-with-resources

Why: try-with-resources closes AutoCloseable resources.

145. A developer wants formatted text such as println/printf rather than raw bytes. Which class fits?

API choice • Java IO

- A. PrintWriter
- B. FileInputStream
- C. Pattern
- D. Matcher

Answer: A. PrintWriter

Why: PrintWriter provides convenient formatted text output.

146. Which statement about Map is correct in the List/Set/Map comparison?

Comparison • Java Collections

- A. it guarantees only unique values and no keys
- B. it stores key-value associations
- C. it is an indexed sequence
- D. it is always a subtype of Collection

Answer: B. it stores key-value associations

Why: Map models key-value associations.

147. A generic method should reject adding arbitrary unrelated objects to a Number producer. Which wildcard is most appropriate for a producer-like view?

Wildcard reasoning • Java Generics

- A. ? super String
- B. ? extends String only
- C. ? extends Number
- D. raw Object

Answer: C. ? extends Number

Why: ? extends Number represents Number or an unknown subtype.

148. Which result is most likely from byte b=2; byte c=3; int x=b+c;?

Code output • Java Numeric

- A. byte 5
- B. short 5
- C. 23
- D. 5

Answer: D. 5

Why: byte operands are promoted to int and 2+3 is 5.

149. What happens to the RHS in (false && (10/0 > 1))?

Exception avoidance • Java Short-circuit

- A. it is not evaluated
- B. division by zero occurs
- C. RHS is evaluated first
- D. expression becomes true

Answer: A. it is not evaluated

Why: false && skips RHS evaluation.

150. What happens to the RHS in (true || (10/0 > 1))?

Exception avoidance • Java Short-circuit

- A. division by zero occurs
- B. it is not evaluated
- C. RHS is evaluated first
- D. expression becomes false

Answer: B. it is not evaluated

Why: true || skips RHS evaluation.

151. Which expression evaluates to 11 under normal Java precedence?

Code output • Java Precedence

- A. $(3 + 2) * 4$
- B. $3 * (2 + 4)$
- C. $3 + 2 * 4$
- D. $3 + 2 + 4 * 2$

Answer: C. $3 + 2 * 4$

Why: Multiplication occurs before addition: $3 + 8 = 11$.

152. A serialized class changes its fields but wants an explicit serialization compatibility identifier. Which declaration is relevant?

API identification • Java Serialization

- A. `transientToken`
- B. `jdbcVersion`
- C. `hashCode`
- D. `serialVersionUID`

Answer: D. `serialVersionUID`

Why: `serialVersionUID` is the serialization compatibility/version identifier.

153. Which field modifier excludes a field from default Java serialization?

Keyword behavior • Java Serialization

- A. `transient`
- B. `volatile`
- C. `final`
- D. `public`

Answer: A. `transient`

Why: `transient` fields are skipped by default serialization.

154. A developer wants to list all paths beneath `/data` recursively as a stream. Which method fits?

Code selection • Java Files

- A. `Files.commit("/data")`
- B. `Files.walk(Path.of("/data"))`
- C. `Pattern.walk("/data")`
- D. `Paths.jdbc("/data")`

Answer: B. `Files.walk(Path.of("/data"))`

Why: `Files.walk` traverses a directory tree and returns a stream of paths.

155. A Java method has overloads `f(int)` and `f(String)`. Calling `f(3)` selects which overload?

Overloading trace • Java OOP

- A. `f(String)`
- B. both equally at runtime
- C. `f(int)`
- D. neither because overloading is runtime-only

Answer: C. `f(int)`

Why: Overload resolution uses the parameter types at compile time.

156. A subclass overrides `run()`. A Parent reference points to Child. Calling `run()` selects what?

Polymorphism trace • Java OOP

- A. `Parent.run()` always
- B. constructor only
- C. the return type chooses
- D. `Child.run()`

Answer: D. `Child.run()`

Why: Overridden instance methods use runtime dispatch.

157. A method inherited as protected is overridden. Which visibility is valid?

Visibility reasoning • Java OOP

- A. public
- B. private only
- C. package-private only if narrower
- D. no visibility allowed

Answer: A. public

Why: An overriding method cannot narrow visibility; public is wider than protected.

158. A class contains an Engine field and delegates start() to it. What relationship is modeled?

Design identification • Java OOP

- A. inheritance/is-a
- B. composition/has-a
- C. overloading
- D. serialization

Answer: B. composition/has-a

Why: Containing another object models has-a composition.

159. A class exposes a private balance field and deposit() validates updates. Which principle is central?

Principle identification • Java OOP

- A. deadlock
- B. overloading
- C. encapsulation
- D. type inference

Answer: C. encapsulation

Why: Private state plus controlled operations is encapsulation.

160. A DataFrame column named score contains nulls. Which expression explicitly keeps only rows with a score?

Null semantics • Spark DataFrames

- A. col("score") != None
- B. col("score") == null
- C. col("score") + 0
- D. col("score").isNotNull()

Answer: D. col("score").isNotNull()

Why: Spark SQL null semantics use isNotNull/isNull.

161. A transformation is built but no action is called. Which statement is safest?

Lazy execution • Spark RDDs

- A. Spark has not fully executed the transformation pipeline yet
- B. every record is already on the driver
- C. the job has committed
- D. the RDD was automatically saved

Answer: A. Spark has not fully executed the transformation pipeline yet

Why: RDD transformations are lazy.

162. An RDD is reused by three actions. Which optimization may avoid recomputing the same upstream work?

Optimization • Spark RDDs

- A. collect every time
- B. persist/cache the reusable RDD
- C. disable lineage
- D. convert it to a regex

Answer: B. persist/cache the reusable RDD

Why: Persistence can keep computed data for reuse.

163. A DataFrame operation requires a new calculated column total = qty * price. Which API is most direct?

Code construction • Spark DataFrames

- A. collect("total")
- B. reduceByKey("total")
- C. withColumn("total", col("qty") * col("price"))
- D. cache("total")

Answer: C. withColumn("total", col("qty") * col("price"))

Why: withColumn adds/replaces a column using an expression.

164. A query selects only customer_id from a wide Parquet table. Which benefit is expected?

Performance • Advanced Spark

- A. JDBC rollback
- B. deadlock prevention
- C. bytecode generation
- D. column pruning can reduce unnecessary column I/O

Answer: D. column pruning can reduce unnecessary column I/O

Why: Parquet supports columnar reads and column pruning.

165. A Parquet dataset is partitioned by year and the query filters year=2025. Which optimization is relevant?

Performance • Advanced Spark

- A. partition pruning
- B. autoboxing
- C. CopyOnWrite
- D. constructor chaining

Answer: A. partition pruning

Why: Filtering partition columns can skip irrelevant partitions.

166. A physical plan contains Exchange around an aggregation. What should a trainee infer first?

Plan diagnosis • Advanced Spark

- A. the query is using Java serialization only
- B. the operation likely involves data redistribution/shuffle
- C. the file is corrupted
- D. the driver has no JVM

Answer: B. the operation likely involves data redistribution/shuffle

Why: Exchange commonly indicates repartitioning/data movement.

167. A built-in lower() function exists for a string transformation. What is the default performance-oriented choice?

Optimization • Advanced Spark

- A. write a UDF automatically
- B. collect to driver first
- C. use the built-in function
- D. use JDBC

Answer: C. use the built-in function

Why: Built-ins are generally easier for Spark to optimize.

168. A UDF is required only for rows passing a selective business rule. Which ordering can reduce UDF work?

Optimization • Advanced Spark

- A. UDF first on every row
- B. collect first
- C. write JSON before filtering
- D. filter first, then apply the UDF

Answer: D. filter first, then apply the UDF

Why: A selective filter reduces the number of rows reaching the expensive UDF.

169. A JDBC source has IDs from 1 to 1,000,000 and a suitable numeric partition column. Which option enables conceptual parallel reads?

Configuration • Advanced Spark/JDBC

- A. partitionColumn with lower/upper bounds and numPartitions
- B. only fetchSize
- C. only a regex
- D. only a watermark

Answer: A. partitionColumn with lower/upper bounds and numPartitions

Why: JDBC partition options allow parallel reads when appropriate.

170. A database can handle only a modest number of concurrent connections. What should guide numPartitions?

Capacity reasoning • Advanced Spark/JDBC

- A. always maximize it
- B. the database's supported concurrency/capacity
- C. always set it to one billion
- D. ignore the source system

Answer: B. the database's supported concurrency/capacity

Why: numPartitions also limits concurrent JDBC partitions/connections.

171. A streaming job must recover progress after a driver restart. Which feature is most relevant?

Recovery • Streaming

- A. List.map
- B. serialVersionUID
- C. durable checkpointing
- D. HashSet

Answer: C. durable checkpointing

Why: Checkpoint metadata supports recovery of query progress/state.

172. A 20-minute event-time window advances every 5 minutes. What does 5 minutes represent?

Parameter identification • Streaming

- A. watermark threshold
- B. window length
- C. checkpoint interval by definition
- D. slide interval

Answer: D. slide interval

Why: The slide determines how often the window result advances.

173. A stream receives events 7 minutes late, and the query should bound late-event state using an event-time threshold. Which mechanism applies?

Lateness handling • Streaming

- A. watermark
- B. cache
- C. Map.put
- D. JDK

Answer: A. watermark

Why: Watermarks help manage lateness and bound event-time state.

174. A team is replacing a legacy RDD-based streaming design with DataFrame-style APIs. Which direction matches the handbook?

Migration concept • Streaming

- A. Structured Streaming -> DStreams
- B. DStreams -> Structured Streaming
- C. RDD -> JDBC only
- D. DataFrame -> Pattern

Answer: B. DStreams -> Structured Streaming

Why: DStream is legacy RDD-based; Structured Streaming uses structured APIs.

175. A transaction must preserve both updates or neither if the second update fails. Which operation should be used on failure?

Failure handling • JDBC

- A. commit()
- B. collect()
- C. rollback()
- D. cache()

Answer: C. rollback()

Why: rollback undoes uncommitted transaction work.

176. A service repeatedly closes and reopens physical database connections. Which mechanism can reduce that overhead?

Performance • JDBC

- A. watermarking
- B. lineage
- C. column pruning
- D. connection pooling

Answer: D. connection pooling

Why: Pooling reuses database connections.

177. Two threads update shared state without coordination and occasionally produce different totals. What should be investigated?

Diagnosis • Concurrency

- A. race condition
- B. Parquet pruning
- C. regex find
- D. JDK compilation

Answer: A. race condition

Why: Timing-dependent shared-state updates indicate a race condition.

178. Two threads acquire two locks in opposite order and stop progressing. What is the likely failure mode?

Diagnosis • Concurrency

- A. autoboxing
- B. deadlock
- C. cache miss
- D. schema inference

Answer: B. deadlock

Why: Opposite lock ordering can create circular wait.

179. A UI has a listener collection with many reads and rare modifications. Which collection matches the stated workload?

Workload selection • Concurrency

- A. write-heavy ArrayList without coordination
- B. JDBC Connection
- C. CopyOnWriteArrayList
- D. Pattern

Answer: C. CopyOnWriteArrayList

Why: CopyOnWriteArrayList is suited to many reads and rare writes.

180. A Java method is declared final and a subclass declares a method with the same signature. What should the trainee expect?

Keyword behavior • Java OOP

- A. the subclass always wins at runtime
- B. both methods are overloaded by return type
- C. the method becomes abstract
- D. the subclass cannot override the final method

Answer: D. the subclass cannot override the final method

Why: A final instance method cannot be overridden.

181. A Java interface reference points to an implementation object. What does this illustrate most directly?

Concept mapping • Java OOP

- A. polymorphism
- B. byte promotion
- C. serialization
- D. file traversal

Answer: A. polymorphism

Why: An interface/parent reference can refer to different concrete implementations.

182. A developer creates two overloaded methods `f(int)` and `f(long)`, then calls `f(3)`. Which one is the direct exact match?

Code reasoning • Java OOP

- A. `f(long)` only because long is wider
- B. `f(int)`
- C. both are selected at runtime
- D. neither can be overloaded

Answer: B. `f(int)`

Why: The int argument exactly matches `f(int)` during overload resolution.

183. A HashMap stores key 'A' with 10, then key 'A' with 25. What should a lookup for 'A' return?

Output prediction • Java HashMap

- A. 10
- B. 35
- C. 25
- D. no value because duplicate keys are forbidden

Answer: C. 25

Why: A new value replaces the existing mapping for an equal key.

184. A developer needs ordered duplicates, but accidentally uses a Set and loses repeated entries. Which collection should be reconsidered?

Debugging • Java Collections

- A. Map only
- B. Iterator only
- C. Pattern
- D. List

Answer: D. List

Why: List is the abstraction for ordered sequences that allow duplicates.

185. A generic collection should prevent inserting a String into a collection intended for Integers at compile time. Which feature supplies that protection?

Type-safety reasoning • Java Generics

- A. parameterized types such as List
- B. transient
- C. serialVersionUID
- D. synchronized

Answer: A. parameterized types such as List

Why: Generics provide compile-time type safety.

186. A developer writes `Number n = 10;` and asks what happened to the primitive 10. Which mechanism may be involved?

Mechanism identification • Java wrappers

- A. unboxing from Number to String
- B. autoboxing from int to Integer
- C. regex conversion
- D. JDBC conversion

Answer: B. autoboxing from int to Integer

Why: Assigning a primitive to a wrapper/reference context can trigger autoboxing.

187. A developer has Integer n = 10; int x = n; Which mechanism converts the wrapper back to a primitive?

Mechanism identification • Java wrappers

- A. autoboxing
- B. serialization
- C. unboxing
- D. pattern matching

Answer: C. unboxing

Why: Unboxing converts a wrapper object to its primitive.

188. A program evaluates (false && risky()) and risky() would divide by zero. Why can this expression avoid the exception?

Exception reasoning • Java Short-circuit

- A. risky() is evaluated twice
- B. Java catches all arithmetic errors
- C. && reverses the operands
- D. risky() is skipped

Answer: D. risky() is skipped

Why: false on the left of && short-circuits the RHS.

189. A program evaluates (true || risky()) and risky() would modify shared state. Why is the state unchanged?

Side-effect reasoning • Java Short-circuit

- A. risky() is skipped
- B. || always evaluates both sides
- C. Java clones the state
- D. the RHS runs only after commit

Answer: A. risky() is skipped

Why: true on the left of || short-circuits the RHS.

190. A developer evaluates 8 + 2 * 5 - 3. What value should they expect?

Code output • Java Precedence

- A. 47
- B. 15
- C. 17
- D. 25

Answer: B. 15

Why: Multiplication is performed first: $8 + 10 - 3 = 15$.

191. A Java object should omit a password field from default serialization. Which modifier is most direct?

Security-oriented behavior • Java Serialization

- A. final
- B. public
- C. transient
- D. volatile

Answer: C. transient

Why: transient excludes the field from default serialized state.

192. A team needs to distinguish incompatible serialized class versions. Which declaration should they understand?

Versioning • Java Serialization

- A. hashCodeOnly
- B. jdbcVersion
- C. threadId
- D. serialVersionUID

Answer: D. serialVersionUID

Why: serialVersionUID is the serialization compatibility/version identifier.

193. A file-copy utility processes an image as raw bytes. Which API family is most appropriate?

API selection • Java IO

- A. byte streams
- B. PrintWriter only
- C. Pattern/Matcher
- D. LocalDate

Answer: A. byte streams

Why: Byte streams handle binary/raw byte data.

194. A file-processing loop uses `BufferedReader` and must close it after use even if parsing fails. Which construct should surround it?

Resource management • Java IO

- A. if-with-resources
- B. try-with-resources
- C. match-with-resources
- D. reduceByKey

Answer: B. try-with-resources

Why: try-with-resources automatically closes managed resources.

195. A regex Pattern is already compiled and the application needs to test a particular input. Which object is created from the Pattern?

API role • Java Regex

- A. ResultSet
- B. HttpClient
- C. Matcher
- D. Iterator

Answer: C. Matcher

Why: Matcher applies a compiled Pattern to input.

196. A developer wants to know whether the entire string 'ABC123' satisfies a pattern, not merely whether a substring does. Which method fits?

Validation • Java Regex

- A. find()
- B. walk()
- C. join()
- D. matches()

Answer: D. matches()

Why: matches attempts whole-input matching.

197. A Scala function receives a list of words and each word should yield its length. Which transformation matches one result per word?

Transformation choice • Scala Functional

- A. map
- B. flatMap
- C. filter
- D. reduceByKey

Answer: A. map

Why: map produces one output for each input.

198. A Scala function receives a list of sentences and each sentence should produce its words, with one final flat list. Which transformation fits?

Transformation choice • Scala Functional

- A. map only
- B. flatMap
- C. filter only
- D. take

Answer: B. flatMap

Why: flatMap maps and flattens nested results.

199. A Spark action is accidentally called on a 500 GB RDD only to inspect five records. Which replacement is safer?

Operational choice • Spark RDDs

- A. collect()
- B. reduceByKey() without keys
- C. take(5)
- D. cache()

Answer: C. take(5)

Why: take retrieves a small bounded sample; collect brings all elements to the driver.

200. A Spark job repeatedly uses an expensive filtered RDD in several actions. Which mechanism is intended to speed reuse?

Optimization • Spark RDDs

- A. collect after every action
- B. transient
- C. watermark
- D. cache/persist

Answer: D. cache/persist

Why: Persistence can avoid recomputing reused RDD work.

SECTION B — 150 SCENARIO-BASED QUESTIONS

Realistic situations covering technical diagnosis, API choice, Spark behavior, performance, recovery and concurrency.

201. A Java inventory object has a private quantity field. An update method rejects negative values before assigning. Which principle is being used?

Scenario-based • Java OOP

- A. encapsulation
- B. inheritance
- C. overloading
- D. deadlock

Answer: A. encapsulation

Why: Private state plus controlled updates is encapsulation.

202. A report generator receives a Shape reference that may refer to Circle or Rectangle and calls area(). Which principle allows the same call to behave differently?

Scenario-based • Java OOP

- A. serialization
- B. polymorphism
- C. constructor identity
- D. type erasure

Answer: B. polymorphism

Why: A parent/interface reference can refer to different concrete implementations.

203. A class's method has the same name as another method but changes only its return type. Can that alone form a valid overload?

Scenario-based • Java OOP

- A. Yes, return type always selects overload
- B. Yes, if method is static
- C. No; changing only return type is not enough
- D. Only if the method is final

Answer: C. No; changing only return type is not enough

Why: Overloading is based on parameter lists, not return type alone.

204. A Java stream pipeline has filter then map, but no terminal operation. What should a trainee expect?

Scenario-based • Java APIs

- A. all elements are immediately printed
- B. filter commits a transaction
- C. map writes a file
- D. the stream pipeline is not fully executed yet

Answer: D. the stream pipeline is not fully executed yet

Why: Java Streams are declarative and terminal operations trigger processing.

205. A developer wants formatted lines into a text file rather than binary bytes. Which API is more suitable?

Scenario-based • Java IO

- A. PrintWriter
- B. FileInputStream
- C. Matcher
- D. ConcurrentHashMap

Answer: A. PrintWriter

Why: PrintWriter is designed for convenient formatted text output.

206. A regex should validate that a six-character code is entirely digits. Which method should be considered?

Scenario-based • Java Regex

- A. find()
- B. matches()
- C. walk()
- D. send()

Answer: B. matches()

Why: matches checks the entire input against the pattern.

207. A regex should locate the first digit sequence inside 'Order A-482'. Which method fits?

Scenario-based • Java Regex

- A. matches()
- B. commit()
- C. find()
- D. persist()

Answer: C. find()

Why: find searches for a matching subsequence.

208. An application must preserve a user's local date and time without a zone because the business rule is purely local. Which type fits?

Scenario-based • Java Time

- A. ZonedDateTime
- B. LocalDate only
- C. LocalTime only
- D. LocalDateTime

Answer: D. LocalDateTime

Why: LocalDateTime represents date and time without zone context.

209. A batch process needs only dates, not times. Which java.time type is the closest fit?

Scenario-based • Java Time

- A. LocalDate
- B. ZonedDateTime
- C. LocalTime
- D. Matcher

Answer: A. LocalDate

Why: LocalDate represents a date without time.

210. A maintenance tool must recursively inspect files under /logs. Which API is aligned?

Scenario-based • Java Files

- A. Pattern.walk(path)
- B. Files.walk(path)
- C. Connection.walk(path)
- D. Matcher.tree(path)

Answer: B. Files.walk(path)

Why: Files.walk traverses a directory tree.

211. A Scala configuration value should never be rebound after initialization. Which keyword should the developer prefer?

Scenario-based • Scala Basics

- A. var
- B. definitelyVar
- C. val
- D. List

Answer: C. val

Why: val creates a fixed binding.

212. A Scala function transforms each input order into one normalized order. Which operation matches one output per input?

Scenario-based • Scala Functional

- A. flatMap
- B. filter only
- C. groupByKey
- D. map

Answer: D. map

Why: map produces one output for each input element.

213. A Scala parser can emit zero, one, or several tokens from each line and the final result should be flat. Which operation fits?

Scenario-based • Scala Functional

- A. flatMap
- B. map
- C. filter
- D. take

Answer: A. flatMap

Why: flatMap maps and flattens nested outputs.

214. A Scala pipeline needs to retain only premium customers. Which operation should be applied for the selection stage?

Scenario-based • Scala Functional

- A. map
- B. filter
- C. flatMap
- D. sorted

Answer: B. filter

Why: filter retains matching elements.

215. A Scala function always returns the same result for the same explicit input and reads no mutable global state. How should it be classified?

Scenario-based • Scala Functional

- A. impure function
- B. side-effecting global state
- C. pure function
- D. race condition

Answer: C. pure function

Why: Purity means deterministic output from explicit inputs without hidden side effects.

216. A Spark RDD is created from a small Seq of test records on the driver. Which API is appropriate?

Scenario-based • Spark RDDs

- A. textFile
- B. join
- C. watermark
- D. parallelize

Answer: D. parallelize

Why: parallelize converts a local collection into an RDD.

217. A Spark RDD is created from a directory of text logs. Which API is the natural basic choice?

Scenario-based • Spark RDDs

- A. textFile
- B. parallelize on every line manually
- C. reduceByKey
- D. collect

Answer: A. textFile

Why: textFile creates an RDD from external text.

218. A developer writes `rdd.filter(...).map(...)` and expects immediate execution after `map`. What correction is needed?

Scenario-based • Spark RDDs

- A. `map` is always an action
- B. an action is needed to trigger execution
- C. `filter` commits results
- D. Spark executes every line eagerly

Answer: B. an action is needed to trigger execution

Why: Transformations are lazy.

219. A job calls `count()` twice on an uncached expensive RDD. What performance issue may occur?

Scenario-based • Spark RDDs

- A. `count` always deletes the RDD
- B. `count` automatically checkpoints
- C. the upstream computation may be repeated
- D. `count` changes keys

Answer: C. the upstream computation may be repeated

Why: Without persistence, reused lineage may be recomputed for separate actions.

220. A pair RDD has (department, salary) records and needs total salary per department. Which operation is most direct?

Scenario-based • Spark Pair RDDs

- A. `collect()` then print all records
- B. `filter` only
- C. `take(5)`
- D. `reduceByKey(_ + _)`

Answer: D. `reduceByKey(_ + _)`

Why: `reduceByKey` performs per-key aggregation.

221. A pair RDD needs all matching customer records from two datasets based on `customerId`. Which operation fits conceptually?

Scenario-based • Spark Pair RDDs

- A. `join`
- B. `map` only
- C. `cache` only
- D. `take`

Answer: A. `join`

Why: `join` combines matching keys across pair RDDs.

222. A developer uses `groupByKey` for a simple sum and asks why `reduceByKey` is often preferred. What is the best explanation at this depth?

Scenario-based • Spark Pair RDDs

- A. `groupByKey` never shuffles
- B. `reduceByKey` performs the per-key reduction rather than materializing all grouped values as a collection
- C. `reduceByKey` runs only on the driver
- D. `groupByKey` converts to `DataFrame`

Answer: B. `reduceByKey` performs the per-key reduction rather than materializing all grouped values as a collection

Why: The handbook emphasizes `reduceByKey` for per-key associative/commutative reduction.

223. A large RDD is inspected with `collect()` and the driver crashes. What should the trainee change first?

Scenario-based • Spark RDDs

- A. increase collect calls
- B. replace with `toString()`
- C. use a bounded action such as `take(n)` or distributed output instead of `collect()`
- D. use a constructor

Answer: C. use a bounded action such as `take(n)` or distributed output instead of `collect()`

Why: `collect` returns all records to the driver.

224. A reused RDD is expensive to compute but fits an available storage level. Which action may improve later reuse?

Scenario-based • Spark RDDs

- A. collect it every time
- B. disable actions
- C. use regex
- D. persist/cache the RDD

Answer: D. persist/cache the RDD

Why: Persistence avoids repeated recomputation when reused.

225. A DataFrame report needs only product_id, quantity, and revenue from a wide dataset. Which transformation should express the projection?

Scenario-based • Spark DataFrames

- A. select the three required columns
- B. collect the full DataFrame
- C. reduceByKey
- D. rollback

Answer: A. select the three required columns

Why: select projects required columns.

226. A DataFrame must remove records missing a required email field but retain rows with null optional fields. Which pattern is appropriate?

Scenario-based • Spark DataFrames

- A. dropna() on every column unconditionally
- B. dropna(subset=["email"])
- C. collect()
- D. reduceByKey

Answer: B. dropna(subset=["email"])

Why: subset limits null checking to the required field.

227. A DataFrame transformation creates cleaned_amount but the next line still sees the old schema. What likely happened?

Scenario-based • Spark DataFrames

- A. withColumn mutates the original automatically
- B. Spark removed the column
- C. the returned DataFrame was not assigned/used
- D. schema changes only after JDBC commit

Answer: C. the returned DataFrame was not assigned/used

Why: DataFrame transformations return a new DataFrame.

228. A DataFrame join followed by groupBy is unexpectedly slow. What should be inspected first at the covered depth?

Scenario-based • Advanced Spark

- A. serialVersionUID
- B. Pattern.matches
- C. Iterator.remove
- D. the logical/physical execution plan with explain()

Answer: D. the logical/physical execution plan with explain()

Why: explain helps diagnose joins, aggregations and shuffle/Exchange.

229. An execution plan contains Exchange. What is the most useful first interpretation?

Scenario-based • Advanced Spark

- A. data redistribution/shuffle is involved
- B. the query is using only local processing
- C. the file is necessarily corrupt
- D. the driver has no heap

Answer: A. data redistribution/shuffle is involved

Why: Exchange commonly indicates repartitioning/data movement.

230. A custom UDF and a built-in function produce the same output. Which should usually be chosen first for performance?

Scenario-based • Advanced Spark

- A. the UDF because it is always faster
- B. the built-in function
- C. collect then use Java code
- D. write JSON and re-read

Answer: B. the built-in function

Why: Built-ins expose semantics to Spark and are generally easier to optimize.

231. A UDF is expensive and only 5% of records meet a simple filter. What sequence is sensible?

Scenario-based • Advanced Spark

- A. UDF all records first
- B. collect all records
- C. apply the filter before the UDF when logically valid
- D. disable the filter

Answer: C. apply the filter before the UDF when logically valid

Why: Filtering reduces UDF work.

232. A JDBC source contains a suitable numeric ID column and bounds. The team wants parallel reads but must avoid excessive database connections. What should they configure carefully?

Scenario-based • Advanced Spark/JDBC

- A. only a watermark
- B. only serialVersionUID
- C. only regex flags
- D. partitionColumn, lowerBound, upperBound and a sensible numPartitions

Answer: D. partitionColumn, lowerBound, upperBound and a sensible numPartitions

Why: These options enable conceptual partitioned reads; numPartitions also affects concurrent connections.

233. A Parquet table has 50 columns but a report needs only 3. Which optimization should the trainee expect?

Scenario-based • Advanced Spark

- A. column pruning
- B. lock ordering
- C. rollback
- D. autoboxing

Answer: A. column pruning

Why: Selecting only needed Parquet columns can reduce I/O.

234. A Parquet dataset is partitioned by month and the report filters month='2026-09'. Which optimization is relevant?

Scenario-based • Advanced Spark

- A. connection pooling
- B. partition pruning
- C. deadlock prevention
- D. byte arithmetic

Answer: B. partition pruning

Why: Filtering a partition column can skip irrelevant partitions.

235. A raw ingestion source is flexible JSON, while downstream analytics are repetitive and column-oriented. Which format flow best matches the handbook?

Scenario-based • Advanced Spark

- A. keep all stages as raw JSON forever
- B. use a Java Iterator as storage
- C. ingest/validate JSON, then write structured columnar output such as Parquet
- D. convert everything directly to a Java object

Answer: C. ingest/validate JSON, then write structured columnar output such as Parquet

Why: JSON is convenient for raw/semi-structured input; Parquet is efficient for downstream analytics.

236. A new streaming application needs DataFrame-style transformations. Which API should be preferred at the covered level?

Scenario-based • Streaming

- A. DStreams
- B. ConcurrentHashMap
- C. JDBC
- D. Structured Streaming

Answer: D. Structured Streaming

Why: Structured Streaming is the preferred modern DataFrame-style streaming API in the handbook.

237. A streaming query loses its driver process and must recover progress/state. What should the team verify?

Scenario-based • Streaming

- A. durable checkpoint storage
- B. that collect() was called
- C. that every event was sent to a socket only
- D. that List is immutable

Answer: A. durable checkpoint storage

Why: Checkpoint data should survive process/driver failure.

238. A team sets a 15-minute window and a 5-minute slide. A manager thinks 5 means only five minutes of history. What correction is needed?

Scenario-based • Streaming

- A. 5 is always the watermark
- B. 5 minutes is how often the window advances; 15 minutes is the included event history
- C. 15 is always the slide
- D. both are checkpoints

Answer: B. 5 minutes is how often the window advances; 15 minutes is the included event history

Why: Window length and slide interval serve different purposes.

239. Events arrive late relative to event time and state must not grow indefinitely. Which mechanism addresses this?

Scenario-based • Streaming

- A. cache
- B. HashSet
- C. watermark
- D. serialVersionUID

Answer: C. watermark

Why: Watermarks establish a lateness threshold and help bound state.

240. A developer uses a socket source for a production recovery-sensitive pipeline. Which concern from the handbook applies?

Scenario-based • Streaming

- A. socket sources are always faster than durable sources
- B. socket input automatically checkpoints
- C. socket sources are required for JDBC
- D. socket input is ephemeral/demo-oriented and not designed like a durable replayable production source

Answer: D. socket input is ephemeral/demo-oriented and not designed like a durable replayable production source

Why: The handbook calls socket sources demo-oriented because they are ephemeral.

241. A team is migrating a legacy RDD-based streaming application to structured APIs. Which pairing is correct?

Scenario-based • Streaming

- A. DStream -> Structured Streaming
- B. Structured Streaming -> DStream
- C. JDBC -> DStream
- D. Map -> watermark

Answer: A. DStream -> Structured Streaming

Why: DStream is legacy RDD-based; Structured Streaming uses structured APIs.

242. A JDBC transaction performs update A, then update B fails before commit. What should happen if both updates must be undone?

Scenario-based • JDBC

- A. commit immediately
- B. rollback the transaction
- C. close without considering transaction state
- D. cache the connection

Answer: B. rollback the transaction

Why: rollback undoes uncommitted changes.

243. A service commits successfully and then calls rollback. What is the key conceptual point?

Scenario-based • JDBC

- A. rollback always reopens the transaction
- B. commit only caches changes
- C. rollback cannot undo changes that were already committed
- D. rollback converts rows to JSON

Answer: C. rollback cannot undo changes that were already committed

Why: Commit makes transaction changes permanent; rollback applies to uncommitted work.

244. A high-throughput service creates a new database connection for every small request. What design could reduce connection creation overhead?

Scenario-based • JDBC

- A. RDD lineage
- B. watermarking
- C. column pruning
- D. connection pooling

Answer: D. connection pooling

Why: Pooling reuses physical connections.

245. A database team objects to setting numPartitions extremely high. Why is their concern relevant?

Scenario-based • Advanced Spark/JDBC

- A. Spark may open many concurrent JDBC partitions/connections, stressing the database
- B. numPartitions only changes regex behavior
- C. higher is always free
- D. numPartitions disables all parallelism

Answer: A. Spark may open many concurrent JDBC partitions/connections, stressing the database

Why: The handbook notes numPartitions limits concurrent JDBC partitions/connections.

246. A counter is updated by two threads and occasionally misses an increment. Which failure mode is most likely?

Scenario-based • Concurrency

- A. deadlock necessarily
- B. race condition/lost update
- C. column pruning
- D. serialization version mismatch

Answer: B. race condition/lost update

Why: Non-atomic shared read-modify-write can lose updates.

247. Two threads each hold one lock and wait for the other's lock. Neither progresses. What should the team diagnose?

Scenario-based • Concurrency

- A. race condition only
- B. connection pooling
- C. deadlock caused by circular waiting
- D. watermarking

Answer: C. deadlock caused by circular waiting

Why: Circular lock waiting can produce deadlock.

248. A code review finds Thread A acquires X then Y, while Thread B acquires Y then X. What simple prevention should be suggested?

Scenario-based • Concurrency

- A. randomize lock order
- B. never release locks
- C. collect all work to the driver
- D. use a consistent global lock order

Answer: D. use a consistent global lock order

Why: Consistent ordering reduces circular-wait risk.

249. Several worker threads need concurrent map updates. Which structure should be considered instead of a plain HashMap at this conceptual level?

Scenario-based • Concurrency

- A. ConcurrentHashMap
- B. CopyOnWriteArrayList for key/value updates
- C. PrintWriter
- D. Pattern

Answer: A. ConcurrentHashMap

Why: ConcurrentHashMap is designed for concurrent map access/updates.

250. A listener registry is read very frequently but changed only during configuration updates. Which collection is conceptually appropriate?

Scenario-based • Concurrency

- A. write-heavy ArrayList
- B. CopyOnWriteArrayList
- C. HashMap for list positions
- D. JDBC Connection

Answer: B. CopyOnWriteArrayList

Why: CopyOnWriteArrayList fits read-heavy, rare-write workloads.

251. A developer selects CopyOnWriteArrayList for a workload with constant writes and sees poor performance. What explains it?

Scenario-based • Concurrency

- A. the collection forbids reads
- B. the JVM cannot allocate arrays
- C. each modification creates a fresh internal copy, making writes expensive
- D. it automatically serializes JDBC

Answer: C. each modification creates a fresh internal copy, making writes expensive

Why: The handbook explicitly warns that frequent writes are expensive.

252. A workflow has loadDataAsync followed by summarizeAsync and needs the second stage to depend on the first result. Which abstraction supports this composition?

Scenario-based • Concurrency

- A. HashSet
- B. PrintWriter
- C. RDD lineage
- D. CompletableFuture

Answer: D. CompletableFuture

Why: CompletableFuture supports asynchronous computation composition.

253. A Java service stores two million distinct customer IDs and needs uniqueness, not positional access. Which collection requirement points to Set?

Scenario-based • Java Collections

- A. Set
- B. List
- C. Map only
- D. Iterator

Answer: A. Set

Why: Set models uniqueness.

254. A reporting tool needs insertion order and allows the same customer ID to appear multiple times because each occurrence is a transaction. Which collection fits?

Scenario-based • Java Collections

- A. Set
- B. List
- C. Map
- D. Pattern

Answer: B. List

Why: List preserves sequence semantics and permits duplicates.

255. A pricing service needs customerId -> latestPrice. When a customer is updated, the latest price should replace the old one. Which abstraction matches?

Scenario-based • Java HashMap

- A. Set
- B. List
- C. Map
- D. CopyOnWriteArrayList

Answer: C. Map

Why: Map models key-value associations and replaces the mapping for an equal key.

256. A HashSet has two employee objects that represent the same employee, but both remain. Which implementation relationship should be checked?

Scenario-based • Java HashSet

- A. println() and Scanner
- B. constructor names
- C. URI syntax
- D. equals() and hashCode() consistency

Answer: D. equals() and hashCode() consistency

Why: Hash-based uniqueness relies on consistent equals/hashCode.

257. A generic method accepts a List and reads values as Number. Which statement matches the wildcard?

Scenario-based • Java Generics

- A. the list contains Number or an unknown subtype
- B. it must contain only Integer
- C. it can contain unrelated Strings
- D. it must be raw

Answer: A. the list contains Number or an unknown subtype

Why: ? extends Number represents Number or an unknown subtype.

258. A developer wants to remove the element returned by the most recent next() call while iterating. Which API is designed for that?

Scenario-based • Java Iterator

- A. List.removeFuture()
- B. Iterator.remove()
- C. Matcher.remove()
- D. Map.deleteNext()

Answer: B. Iterator.remove()

Why: Iterator.remove removes the last element returned by next().

259. A developer writes short s=2, t=3; short u=s+t; and compilation complains. What source concept explains it?

Scenario-based • Java Numeric

- A. short arithmetic always stays short
- B. shorts become String
- C. the arithmetic result is promoted to int
- D. the result becomes byte

Answer: C. the arithmetic result is promoted to int

Why: byte/short arithmetic is promoted to int.

260. A boolean condition begins with false && a method that increments an audit counter. Why is the audit counter unchanged?

Scenario-based • Java Short-circuit

- A. the JVM resets counters
- B. && executes RHS first
- C. the method becomes synchronized
- D. the RHS method is skipped

Answer: D. the RHS method is skipped

Why: false && short-circuits.

261. A boolean condition begins with true || a method that logs a message. Why might the log never appear?

Scenario-based • Java Short-circuit

- A. the RHS is skipped
- B. true forces RHS evaluation
- C. || reverses the condition
- D. logging is disabled by Java

Answer: A. the RHS is skipped

Why: true || short-circuits.

262. A developer calculates $4 * 5 + 6 / 2$ and gets 23. Which precedence rule supports it?

Scenario-based • Java Precedence

- A. addition occurs first
- B. multiplication and division occur before addition
- C. all operators have equal precedence
- D. division occurs last

Answer: B. multiplication and division occur before addition

Why: $4 * 5 + 6 / 2 = 20 + 3 = 23$.

263. A Serializable object has a transient cache field. After deserialization, the cache is not restored by default. Why?

Scenario-based • Java Serialization

- A. transient encrypts it
- B. transient makes it static
- C. transient excludes the field from default serialized state
- D. transient stores it in JDBC

Answer: C. transient excludes the field from default serialized state

Why: Default serialization skips transient fields.

264. A file service needs raw PNG bytes, not formatted characters. Which family should be used?

Scenario-based • Java IO

- A. PrintWriter
- B. Matcher
- C. LocalDate
- D. byte streams

Answer: D. byte streams

Why: Byte streams are intended for binary/raw bytes.

265. A file service writes human-readable lines using printf-style formatting. Which class fits?

Scenario-based • Java IO

- A. PrintWriter
- B. FileInputStream
- C. Pattern
- D. HttpClient

Answer: A. PrintWriter

Why: PrintWriter supports convenient formatted text output.

266. A regex must reject 'ABC123x' because the extra x is outside the intended six-character code. Which method should validate the whole input?

Scenario-based • Java Regex

- A. find()
- B. matches()
- C. walk()
- D. groupOnly()

Answer: B. matches()

Why: matches validates the entire input.

267. A regex is used to locate '2026' somewhere in 'build-2026-final'. Which method is more appropriate?

Scenario-based • Java Regex

- A. matches()
- B. commit()
- C. find()
- D. persist()

Answer: C. find()

Why: find locates a matching subsequence.

268. A payment event must retain its 2026-10-07 date, 09:30 time, and Asia/Kolkata zone. Which type is appropriate?

Scenario-based • Java Time

- A. LocalDate
- B. LocalTime
- C. LocalDateTime
- D. ZonedDateTime

Answer: D. ZonedDateTime

Why: ZonedDateTime includes date, time, and zone context.

269. A batch job only records 2026-10-07 and no time-of-day. Which type is most direct?

Scenario-based • Java Time

- A. LocalDate
- B. LocalTime
- C. ZonedDateTime
- D. Matcher

Answer: A. LocalDate

Why: LocalDate represents date without time.

270. A Scala counter changes each iteration of a loop. Which binding supports reassignment?

Scenario-based • Scala Basics

- A. val
- B. var
- C. Tuple2
- D. List

Answer: B. var

Why: var is reassignable.

271. A Scala configuration object should not be rebound, although the object itself may have its own semantics. Which keyword fixes the binding?

Scenario-based • Scala Basics

- A. var
- B. def
- C. val
- D. RDD

Answer: C. val

Why: val cannot be rebound.

272. A Scala expression produces (101, "IN", true). What is the most natural type category?

Scenario-based • Scala Basics

- A. Set only
- B. homogeneous List only
- C. JDBC ResultSet
- D. tuple

Answer: D. tuple

Why: Tuples hold a fixed heterogeneous group.

273. A Scala program matches status strings with cases "NEW", "PAID", and a default. Which control mechanism is intended?

Scenario-based • Scala Basics

- A. pattern matching
- B. try-with-resources
- C. JDBC commit
- D. HashSet

Answer: A. pattern matching

Why: Scala match uses ordered case patterns.

274. A case matches an integer but should only run when it is positive. Which construct adds that condition?

Scenario-based • Scala Basics

- A. tuple
- B. pattern guard
- C. flatMap
- D. serialVersionUID

Answer: B. pattern guard

Why: A guard adds an if-condition to a case.

275. A Scala pipeline receives List(1,2,3) and should return List(2,4,6). Which operation is the natural first choice?

Scenario-based • Scala Functional

- A. flatMap
- B. filter
- C. map
- D. groupByKey

Answer: C. map

Why: map transforms each element one-for-one.

276. A Scala pipeline receives List("a,b", "c") and should produce a flat list of tokens. Which operation is natural?

Scenario-based • Scala Functional

- A. map only
- B. filter
- C. take
- D. flatMap

Answer: D. flatMap

Why: flatMap can emit multiple tokens per input and flatten them.

277. A Scala function receives an age and should retain only ages ≥ 18 . Which operation is the direct selection step?

Scenario-based • Scala Functional

- A. filter
- B. map
- C. flatMap
- D. sorted

Answer: A. filter

Why: filter retains matching elements.

278. A pure Scala function uses only its arguments and no mutable global state. Why is it useful in distributed processing?

Scenario-based • Scala Functional

- A. it requires shared state
- B. its result is deterministic from explicit inputs
- C. it depends on current time
- D. it must mutate a collection

Answer: B. its result is deterministic from explicit inputs

Why: Purity reduces hidden side effects and improves reasoning.

279. A Spark test creates `sc.parallelize(Seq("a","b","c"), 2)`. What does the second argument represent conceptually?

Scenario-based • Spark RDDs

- A. the number of actions
- B. the driver memory limit
- C. the requested partition count
- D. the JDBC pool size

Answer: C. the requested partition count

Why: parallelize can specify a partition count.

280. A Spark developer uses `textFile` for a local in-memory Seq. What is the conceptual mismatch?

Scenario-based • Spark RDDs

- A. `textFile` is only for JDBC
- B. `parallelize` is only for Parquet
- C. both APIs always mean the same thing
- D. `textFile` is for external text sources; `parallelize` is for local collections

Answer: D. `textFile` is for external text sources; `parallelize` is for local collections

Why: The handbook distinguishes local collection -> `parallelize` from external text source -> `textFile`.

281. A Spark transformation is defined but the job has no action. Why is cluster work minimal?

Scenario-based • Spark RDDs

- A. transformations are lazy
- B. transformations are database commits
- C. RDDs always execute immediately
- D. the driver is blocked forever

Answer: A. transformations are lazy

Why: Execution is triggered by actions.

282. A Spark job loses a partition of an RDD derived by `map/filter`. What lets Spark reason about recomputing it?

Scenario-based • Spark RDDs

- A. Map key uniqueness
- B. lineage
- C. `serialVersionUID`
- D. watermark

Answer: B. lineage

Why: Lineage records transformation dependencies.

283. A developer needs only ten records for debugging from a huge RDD. Which action avoids loading the entire RDD to the driver?

Scenario-based • Spark RDDs

- A. `collect()`
- B. `groupByKey()`
- C. `take(10)`
- D. `cache()`

Answer: C. `take(10)`

Why: `take` returns a bounded number of elements.

284. A pair RDD contains `("IN",100)`, `("IN",50)`, `("US",70)`. What does `reduceByKey(_ + _)` produce for IN?

Scenario-based • Spark Pair RDDs

- A. IN -> 5000
- B. IN -> 50
- C. IN -> 100
- D. IN -> 150

Answer: D. IN -> 150

Why: `reduceByKey` adds values sharing the same key.

285. A pair RDD contains `customerId` -> order records and another contains `customerId` -> profile records. Which operation combines matching IDs?

Scenario-based • Spark Pair RDDs

- A. `join`
- B. `filter`
- C. `cache`
- D. `collect`

Answer: A. `join`

Why: `join` combines matching keys.

286. A Spark engineer says groupByKey is required for every aggregation. What should the trainee challenge?

Scenario-based • Spark Pair RDDs

- A. groupByKey is the only Spark operation
- B. simple associative per-key reductions can use reduceByKey
- C. reduceByKey cannot aggregate
- D. groupByKey never shuffles

Answer: B. simple associative per-key reductions can use reduceByKey

Why: The handbook specifically emphasizes reduceByKey for per-key aggregation.

287. A DataFrame has 30 columns, but the report uses only name and revenue. Which operation expresses the needed projection?

Scenario-based • Spark DataFrames

- A. collect()
- B. reduceByKey()
- C. select("name", "revenue")
- D. checkpoint()

Answer: C. select("name", "revenue")

Why: select projects requested columns.

288. A DataFrame needs only records with amount >= 1000. Which transformation expresses that?

Scenario-based • Spark DataFrames

- A. cache()
- B. collect()
- C. withColumn("amount") only
- D. filter(col("amount") >= 1000)

Answer: D. filter(col("amount") >= 1000)

Why: filter selects rows satisfying a predicate.

289. A DataFrame needs a derived column tax = amount * 0.18. Which API fits?

Scenario-based • Spark DataFrames

- A. withColumn("tax", col("amount") * 0.18)
- B. collect("tax")
- C. reduceByKey("tax")
- D. Iterator.remove()

Answer: A. withColumn("tax", col("amount") * 0.18)

Why: withColumn creates a derived/replaced column.

290. A DataFrame has a nullable rating field and the requirement is to retain only rows where rating exists. Which check is explicit?

Scenario-based • Spark DataFrames

- A. != None
- B. isNotNull()
- C. == null
- D. collect()

Answer: B. isNotNull()

Why: isNotNull is the explicit Spark SQL null check.

291. A DataFrame join is unexpectedly expensive. Which command helps inspect how Spark plans the operation?

Scenario-based • Advanced Spark

- A. Pattern.compile()
- B. Files.walk()
- C. df.explain()
- D. Integer.valueOf()

Answer: C. df.explain()

Why: explain displays execution-plan information.

292. An explain plan contains Exchange after a groupBy. What does the trainee infer?

Scenario-based • Advanced Spark

- A. the DataFrame became an RDD automatically
- B. the database committed
- C. a regex matched
- D. data redistribution/shuffle is occurring

Answer: D. data redistribution/shuffle is occurring

Why: Exchange commonly indicates repartitioning/data movement.

293. A built-in regexp_replace can solve a cleaning task currently implemented with a UDF. Which choice generally exposes more information to Spark?

Scenario-based • Advanced Spark

- A. the built-in function
- B. the opaque UDF
- C. collect and process in driver
- D. serialize each row first

Answer: A. the built-in function

Why: Built-ins are generally easier for Spark to optimize.

294. A UDF takes 2 seconds per row, but a cheap filter can remove 95% of rows. What should be considered?

Scenario-based • Advanced Spark

- A. UDF first on every row
- B. filter first when the filter is logically safe
- C. collect all rows
- D. disable predicate filtering

Answer: B. filter first when the filter is logically safe

Why: Filtering first reduces UDF invocations.

295. A JDBC source has IDs 1-10 million and can handle 8 concurrent connections. What should guide a reasonable partition configuration?

Scenario-based • Advanced Spark/JDBC

- A. set numPartitions to millions
- B. ignore database capacity
- C. use a suitable partition column/bounds and avoid numPartitions far beyond the database's capacity
- D. use watermark instead

Answer: C. use a suitable partition column/bounds and avoid numPartitions far beyond the database's capacity

Why: The handbook stresses sensible partitioning and database capacity.

296. A Parquet query projects 4 columns from a 100-column dataset. Why can this be efficient?

Scenario-based • Advanced Spark

- A. Parquet forces all columns to be read
- B. JDBC rollback occurs
- C. locks are acquired
- D. column pruning can reduce the amount of data read

Answer: D. column pruning can reduce the amount of data read

Why: Parquet is columnar and supports column pruning.

297. A partitioned Parquet dataset has year and region partitions. The query filters both. What can happen?

Scenario-based • Advanced Spark

- A. partition pruning can skip irrelevant partitions
- B. all partitions must be read
- C. a deadlock is created
- D. JDBC pooling starts

Answer: A. partition pruning can skip irrelevant partitions

Why: Partition filters can enable partition pruning.

298. A raw ingestion layer receives flexible JSON with evolving fields, but analytics repeatedly scans stable columns. Which downstream format is better aligned?

Scenario-based • Advanced Spark

- A. socket stream only
- B. Parquet
- C. PrintWriter only
- D. Java Iterator

Answer: B. Parquet

Why: The handbook describes JSON as convenient for raw ingestion and Parquet as efficient for analytics.

299. A Structured Streaming query processes event-time windows and needs to tolerate some lateness. Which two concepts should the trainee connect?

Scenario-based • Streaming

- A. autoboxing and serialVersionUID
- B. JDBC pooling and Map
- C. watermark and event-time windowing
- D. List and Set

Answer: C. watermark and event-time windowing

Why: Watermarks manage lateness and bound event-time state.

300. A streaming query is restarted after a driver failure and must resume progress. What storage should be durable?

Scenario-based • Streaming

- A. only the Java stack
- B. only the current UI
- C. only a local temporary socket
- D. checkpoint data

Answer: D. checkpoint data

Why: Durable checkpointing supports recovery.

301. A 1-hour window slides every 15 minutes. How many minutes of history does each window cover?

Scenario-based • Streaming

- A. 60 minutes
- B. 15 minutes
- C. 75 minutes
- D. 4 minutes

Answer: A. 60 minutes

Why: Window length is the amount of event history included.

302. A 1-hour window slides every 15 minutes. How often does the window result advance?

Scenario-based • Streaming

- A. 60 minutes
- B. 15 minutes
- C. 75 minutes
- D. 4 hours

Answer: B. 15 minutes

Why: Slide interval controls how often the result advances.

303. A stream receives records that are 20 minutes late, and the business allows only 10 minutes of lateness. Which concept encodes that threshold?

Scenario-based • Streaming

- A. cache
- B. connection pool
- C. watermark
- D. List

Answer: C. watermark

Why: A watermark expresses the lateness threshold for event-time processing.

304. A developer uses DStreams for a new structured streaming pipeline and asks whether there is a newer structured approach. What should be recommended at this assessment depth?

Scenario-based • Streaming

- A. DStreams only
- B. ConcurrentHashMap
- C. JDBC
- D. Structured Streaming

Answer: D. Structured Streaming

Why: The handbook positions Structured Streaming as the preferred API for new DataFrame-style streaming.

305. A socket stream loses data when the demo process restarts. Why is this not surprising?

Scenario-based • Streaming

- A. socket input is ephemeral and not designed for durable replay/recovery
- B. socket input is always durable
- C. watermarks guarantee source replay
- D. DStreams persist every byte automatically

Answer: A. socket input is ephemeral and not designed for durable replay/recovery

Why: The handbook explicitly characterizes socket sources as demo-oriented.

306. A transaction has inserted three rows and updated two more, but a validation failure occurs before commit. The requirement is no partial change. What should happen?

Scenario-based • JDBC

- A. commit()
- B. rollback()
- C. cache()
- D. collect()

Answer: B. rollback()

Why: rollback undoes uncommitted work.

307. A developer calls commit after every single row update and performance suffers. What conceptual concern does this illustrate?

Scenario-based • JDBC

- A. commit is the same as rollback
- B. commit creates RDD lineage
- C. transaction boundaries can create excessive commit overhead
- D. commit is a regex operation

Answer: C. transaction boundaries can create excessive commit overhead

Why: The handbook covers commit/rollback conceptually; grouping work into appropriate transactions can avoid excessive transaction overhead.

308. A service has 100 requests per second and each request creates a new physical DB connection. What mechanism should be evaluated?

Scenario-based • JDBC

- A. watermark
- B. Parquet pruning
- C. Pattern
- D. connection pooling

Answer: D. connection pooling

Why: Pooling reuses physical database connections.

309. A DBA sees 40 simultaneous connections during a Spark JDBC load. Which Spark JDBC concept should be reviewed?

Scenario-based • Advanced Spark/JDBC

- A. numPartitions
- B. serialVersionUID
- C. slide interval
- D. transient

Answer: A. numPartitions

Why: numPartitions also limits concurrent JDBC partitions/connections.

310. Two threads update a shared inventory count. Occasionally stock becomes one unit too high or low depending on timing. What should be diagnosed?

Scenario-based • Concurrency

- A. deadlock necessarily
- B. race condition
- C. column pruning
- D. serialization version mismatch

Answer: B. race condition

Why: Timing-dependent shared mutable state results indicate a race.

311. Thread A holds lock X and waits for Y; Thread B holds Y and waits for X. What should a reviewer call this?

Scenario-based • Concurrency

- A. pure function
- B. cache hit
- C. deadlock
- D. watermark

Answer: C. deadlock

Why: This is circular waiting.

312. A team standardizes all lock acquisition to X then Y across all code paths. What risk are they reducing?

Scenario-based • Concurrency

- A. autoboxing
- B. column pruning
- C. regex mismatch
- D. deadlock due to circular wait

Answer: D. deadlock due to circular wait

Why: Consistent lock ordering reduces circular waiting.

313. A service needs a concurrent map for multiple worker threads. Why is ConcurrentHashMap a better conceptual choice than plain HashMap without coordination?

Scenario-based • Concurrency

- A. it is designed for concurrent map access and updates
- B. it guarantees read-only data
- C. it is a streaming source
- D. it is a regex engine

Answer: A. it is designed for concurrent map access and updates

Why: ConcurrentHashMap is the covered concurrent map structure.

314. A listener registry has 50,000 reads per second and a handful of writes per hour. Which collection matches the workload?

Scenario-based • Concurrency

- A. write-heavy ArrayList
- B. CopyOnWriteArrayList
- C. JDBC Connection
- D. Pattern

Answer: B. CopyOnWriteArrayList

Why: CopyOnWriteArrayList suits many reads and rare writes.

315. A CopyOnWriteArrayList is modified thousands of times per second. Which trade-off explains poor performance?

Scenario-based • Concurrency

- A. reads become impossible
- B. all writes are sent to JDBC
- C. writes are expensive because modifications copy the underlying array
- D. it always deadlocks

Answer: C. writes are expensive because modifications copy the underlying array

Why: The handbook explicitly notes the cost of frequent writes.

316. A service starts loadData asynchronously and needs to transform the result after completion. Which abstraction supports that?

Scenario-based • Concurrency

- A. HashMap only
- B. PrintWriter
- C. Iterator
- D. CompletableFuture

Answer: D. CompletableFuture

Why: CompletableFuture represents an asynchronous computation that can be chained.

317. A CompletableFuture chain must combine two independent async results before producing a report. Which high-level capability matters?

Scenario-based • Concurrency

- A. asynchronous composition/combination
- B. RDD checkpointing
- C. regex find
- D. JDBC rollback

Answer: A. asynchronous composition/combination

Why: CompletableFuture supports chaining and combining asynchronous computations.

318. A Java class has a parent public method and the child tries to override it with protected visibility. Which issue should be identified?

Scenario-based • Java OOP

- A. the child is widening visibility
- B. the child is narrowing visibility
- C. the method is overloaded only
- D. constructors are involved

Answer: B. the child is narrowing visibility

Why: Overriding cannot reduce visibility.

319. A Java method has overloads f(int) and f(double). The call f(2) is made. Which overload is the exact parameter match?

Scenario-based • Java OOP

- A. f(double) only
- B. both at runtime
- C. f(int)
- D. neither

Answer: C. f(int)

Why: Overloading is resolved from parameter types at compile time.

320. A Scala list of 1,2,3 is transformed with filter(_ > 1). What remains?

Scenario-based • Scala Functional

- A. 1 only
- B. 1,2,3
- C. 3 only
- D. 2 and 3

Answer: D. 2 and 3

Why: filter retains values satisfying the predicate.

321. A Scala list of strings is transformed with map(_.length). What kind of output is produced?

Scenario-based • Scala Functional

- A. one integer result per input string
- B. zero outputs always
- C. multiple flattened outputs per string
- D. only the longest string

Answer: A. one integer result per input string

Why: map produces one output per input.

322. A Spark RDD uses map and then filter but no action. What is stored conceptually?

Scenario-based • Spark RDDs

- A. all results in driver memory
- B. lazy transformation lineage
- C. a committed database transaction
- D. a completed output file

Answer: B. lazy transformation lineage

Why: Transformations are lazy and build lineage.

323. A Spark pair RDD needs the keys only for a downstream operation. Which API extracts them?

Scenario-based • Spark Pair RDDs

- A. values()
- B. collect()
- C. keys()
- D. cache()

Answer: C. keys()

Why: keys() extracts the key component.

324. A Spark pair RDD needs only the values for a downstream numeric calculation. Which API extracts them?

Scenario-based • Spark Pair RDDs

- A. keys()
- B. join()
- C. checkpoint()
- D. values()

Answer: D. values()

Why: values() extracts the value component.

325. A DataFrame has an optional nickname column and a required user_id column. Which null-removal pattern targets only user_id?

Scenario-based • Spark DataFrames

- A. dropna(subset=["user_id"])
- B. dropna() on every column
- C. collect()
- D. filter(user_id == null)

Answer: A. dropna(subset=["user_id"])

Why: subset limits the null check to the required field.

326. A DataFrame calculation is written with withColumn, but the developer expects the original DataFrame object to change automatically. What should they remember?

Scenario-based • Spark DataFrames

- A. withColumn is always an action
- B. the transformation returns a new DataFrame
- C. DataFrames are mutable objects
- D. Spark updates every reference

Answer: B. the transformation returns a new DataFrame

Why: DataFrame transformations return new DataFrames.

327. A Spark plan contains Exchange near a join. What should the trainee investigate?

Scenario-based • Advanced Spark

- A. Java autoboxing
- B. regex matching
- C. data redistribution/shuffle
- D. file closing

Answer: C. data redistribution/shuffle

Why: Exchange commonly indicates repartitioning/data movement.

328. A Parquet query selects one column from a wide table. Which physical-data benefit may occur?

Scenario-based • Advanced Spark

- A. connection pooling
- B. deadlock prevention
- C. rollback
- D. column pruning

Answer: D. column pruning

Why: Column pruning reduces unnecessary column reads.

329. A streaming query's state keeps growing because late events are accepted indefinitely. Which concept should be reviewed?

Scenario-based • Streaming

- A. watermark
- B. serialVersionUID
- C. HashMap
- D. commit

Answer: A. watermark

Why: Watermarks help bound event-time state.

330. A JDBC transaction must undo uncommitted changes after a validation failure. Which operation fits?

Scenario-based • JDBC

- A. commit()
- B. rollback()
- C. cache()
- D. persist()

Answer: B. rollback()

Why: rollback undoes uncommitted changes.

331. A concurrent map is read and updated by many threads. Which covered collection should be considered?

Scenario-based • Concurrency

- A. Pattern
- B. PrintWriter
- C. ConcurrentHashMap
- D. LocalDate

Answer: C. ConcurrentHashMap

Why: ConcurrentHashMap is designed for concurrent map access.

332. A concurrent listener list is almost always read and rarely modified. Which covered collection matches?

Scenario-based • Concurrency

- A. HashMap
- B. FileInputStream
- C. JDBC Connection
- D. CopyOnWriteArrayList

Answer: D. CopyOnWriteArrayList

Why: CopyOnWriteArrayList suits read-heavy, rare-write workloads.

333. A developer calls collect() on an RDD containing 266 GB only to inspect a handful of records. Which safer approach matches the handbook?

Scenario-based • Spark RDDs

- A. use a bounded action such as take(n)
- B. collect() again
- C. groupByKey()
- D. cache() without limiting the result

Answer: A. use a bounded action such as take(n)

Why: collect brings the entire RDD to the driver; take retrieves a bounded sample.

334. A Parquet report reads only 3 columns from a table containing 174 columns. Which optimization should the trainee identify?

Scenario-based • Advanced Spark

- A. deadlock prevention
- B. column pruning
- C. JDBC rollback
- D. autoboxing

Answer: B. column pruning

Why: Selecting only needed Parquet columns can reduce I/O.

335. Two threads both read a shared counter at 135, compute 136, and write it back. What failure mode can cause a lost increment?

Scenario-based • Concurrency

- A. deadlock necessarily
- B. column pruning
- C. race condition
- D. connection pooling

Answer: C. race condition

Why: A non-atomic shared read-modify-write can produce timing-dependent lost updates.

336. A Scala pipeline receives 136 customer records and should keep only those above an eligibility threshold before an expensive transformation. Which first-stage operation is most direct?

Scenario-based • Scala Functional

- A. map
- B. flatMap
- C. reduceByKey
- D. filter

Answer: D. filter

Why: filter is the direct selection operation and can reduce later work.

337. A Java application serializes a user profile but should not persist a one-time authentication token across sessions. Which field modifier addresses the default serialization behavior?

Scenario-based • Java Serialization

- A. transient
- B. final
- C. volatile
- D. public

Answer: A. transient

Why: transient excludes the field from default serialized state.

338. A DataFrame contains a required customer_id and an optional nickname. The pipeline should remove rows only when customer_id is missing. Which operation is most targeted?

Scenario-based • Spark DataFrames

- A. dropna() across every column
- B. dropna(subset=["customer_id"])
- C. collect()
- D. col("customer_id") != None

Answer: B. dropna(subset=["customer_id"])

Why: subset limits null handling to the specified required column.

339. A JDBC transaction has successfully performed several changes, but a business validation fails before commit and the changes must be discarded as one unit. Which operation matches the requirement?

Scenario-based • JDBC

- A. commit()
- B. cache()
- C. rollback()
- D. persist()

Answer: C. rollback()

Why: rollback undoes uncommitted transaction work.

340. A Spark job processes 140,000 rows, keeps only 2% after a cheap predicate, and then runs an expensive custom calculation. Which ordering is most sensible?

Scenario-based • Advanced Spark

- A. run the custom calculation on every row first
- B. collect all rows to the driver
- C. disable the predicate
- D. apply the selective filter before the expensive custom calculation when logically valid

Answer: D. apply the selective filter before the expensive custom calculation when logically valid

Why: Filtering early reduces the number of rows reaching expensive custom logic.

341. A streaming query uses a 141 minute window and a 47 minute slide. Which value describes how often the window result advances?

Scenario-based • Streaming

- A. 47 minutes
- B. 141 minutes
- C. the watermark duration
- D. the checkpoint retention

Answer: A. 47 minutes

Why: The slide interval controls how often the window advances.

342. A JDBC load is configured with numPartitions=6, but the DBA says the database supports only a small number of concurrent connections. What should be reviewed?

Scenario-based • Advanced Spark/JDBC

- A. always maximize numPartitions
- B. the JDBC partition count should be kept within the database's supported concurrency
- C. replace JDBC with regex
- D. ignore the database capacity

Answer: B. the JDBC partition count should be kept within the database's supported concurrency

Why: numPartitions also affects concurrent JDBC partitions/connections.

343. A developer calls collect() on an RDD containing 286 GB only to inspect a handful of records. Which safer approach matches the handbook?

Scenario-based • Spark RDDs

- A. collect() again
- B. groupByKey()
- C. use a bounded action such as take(n)
- D. cache() without limiting the result

Answer: C. use a bounded action such as take(n)

Why: collect brings the entire RDD to the driver; take retrieves a bounded sample.

344. A Parquet report reads only 3 columns from a table containing 184 columns. Which optimization should the trainee identify?

Scenario-based • Advanced Spark

- A. deadlock prevention
- B. JDBC rollback
- C. autoboxing
- D. column pruning

Answer: D. column pruning

Why: Selecting only needed Parquet columns can reduce I/O.

345. Two threads both read a shared counter at 145, compute 146, and write it back. What failure mode can cause a lost increment?

Scenario-based • Concurrency

- A. race condition
- B. deadlock necessarily
- C. column pruning
- D. connection pooling

Answer: A. race condition

Why: A non-atomic shared read-modify-write can produce timing-dependent lost updates.

346. A Scala pipeline receives 146 customer records and should keep only those above an eligibility threshold before an expensive transformation. Which first-stage operation is most direct?

Scenario-based • Scala Functional

- A. map
- B. filter
- C. flatMap
- D. reduceByKey

Answer: B. filter

Why: filter is the direct selection operation and can reduce later work.

347. A Java service must omit a temporary token field from default serialization. Which modifier should the trainee use?

Scenario-based • Java Serialization

- A. public
- B. volatile
- C. transient
- D. synchronized

Answer: C. transient

Why: transient excludes a field from default serialized state.

348. A DataFrame column 'rating' may be null, but the business rule requires only rows where rating exists. Which explicit predicate should be used?

Scenario-based • Spark DataFrames

- A. col("rating") != None
- B. col("rating") == null
- C. collect()
- D. col("rating").isNotNull()

Answer: D. col("rating").isNotNull()

Why: isNotNull explicitly handles Spark SQL null semantics.

349. A transaction has made several changes but must undo them because validation failed before the final commit. Which JDBC operation is correct?

Scenario-based • JDBC

- A. rollback()
- B. commit()
- C. cache()
- D. persist()

Answer: A. rollback()

Why: rollback undoes uncommitted transaction work.

350. A Spark job processes 150,000 rows, keeps only 2% after a cheap predicate, and then runs an expensive custom calculation. Which ordering is most sensible?

Scenario-based • Advanced Spark

- A. run the custom calculation on every row first
- B. apply the selective filter before the expensive custom calculation when logically valid
- C. collect all rows to the driver
- D. disable the predicate

Answer: B. apply the selective filter before the expensive custom calculation when logically valid

Why: Filtering early reduces the number of rows reaching expensive custom logic.